MODULE 06

Supervision d'un parking souterrain : ajout de nouveaux capteurs simulés

Gaëtan GEORGES
Pascal HAGOT
David IERVESE

Lycée Louis Armand
173 Bd de Strasbourg
Nogent sur Marne

MODULE 06

SÉANCE SYSTÈME 01
TP D'INFORMATIQUE

Durée 2h30

DÉCOUVERTE ET INSTALLATION DU SYSTÈME CIRPARK : INTÉGRATION DE CAPTEURS SIMULÉS

BLOC DE COMPÉTENCES

U5 - EXPLOITATION ET MAINTENANCE DE RÉSEAUX INFORMATIQUES

COMPÉTENCE(S)

C09 - INSTALLER UN RÉSEAU INFORMATIQUE

OBJECTIF PÉDAGOGIQUE

Installation et mise en œuvre d'un système de supervision de parking souterrain. Utilisation d'un client REST pour tester les possibilités offertes par l'API HTTP(données au format XML). Étude de la communication entre les différents éléments.

CONNAISSANCES ISSUES DU RÉFÉRENTIEL

• Réseaux de terrain	Niveau 3
• Réseaux informatiques (protocoles, équipements et outils usuels et industriels)	Niveau 3
• Outils logiciels d'évaluation, de traçabilité de l'information	Niveau 3

CONNAISSANCES OPÉRATIONNALISÉES

• Installer et configurer un matériel à partir d'une documentation	Niveau 3
• Analyser une communication sur port série virtuel	Niveau 2
• Analyser une communication réseau avec Wireshark	Niveau 3

Gaëtan GEORGES
Pascal HAGOT
David IERVESE

Lycée Louis Armand
173 Bd de Strasbourg
Nogent sur Marne

TD
Découverte du système Cirpark

Mise en situation : parking intelligent cirpark

Cirpark est une solution qui permet la gestion intelligente d'un parking. Cette solution s'appuie sur un réseau de capteurs qui permettent la détection de présence de véhicules aux emplacements réservés. Un système de guidage oriente l'utilisateur du parking vers les places libres (affichages dans le parking, application mobile). Une centrale de supervision permet de suivre en temps réel l'état du parking ainsi que des alarmes techniques (Monoxyde de carbone par exemple). La solution est extensible vers la gestion des alarmes techniques, d'un système de réservation associé au numéro de plaques des véhicules (reconnaissance de forme), gestion de barrière, paiement…

Une fois la configuration du parking finalisée elle est publiée sur le serveur web (Engine) à l'aide d'une fonction d'exportation.

Le serveur assure les services suivant :

- Gestion du parc de capteurs, détecteurs et afficheurs.
- Service web (données xml) permettant à une application tierce d'exploiter les données de l'application.

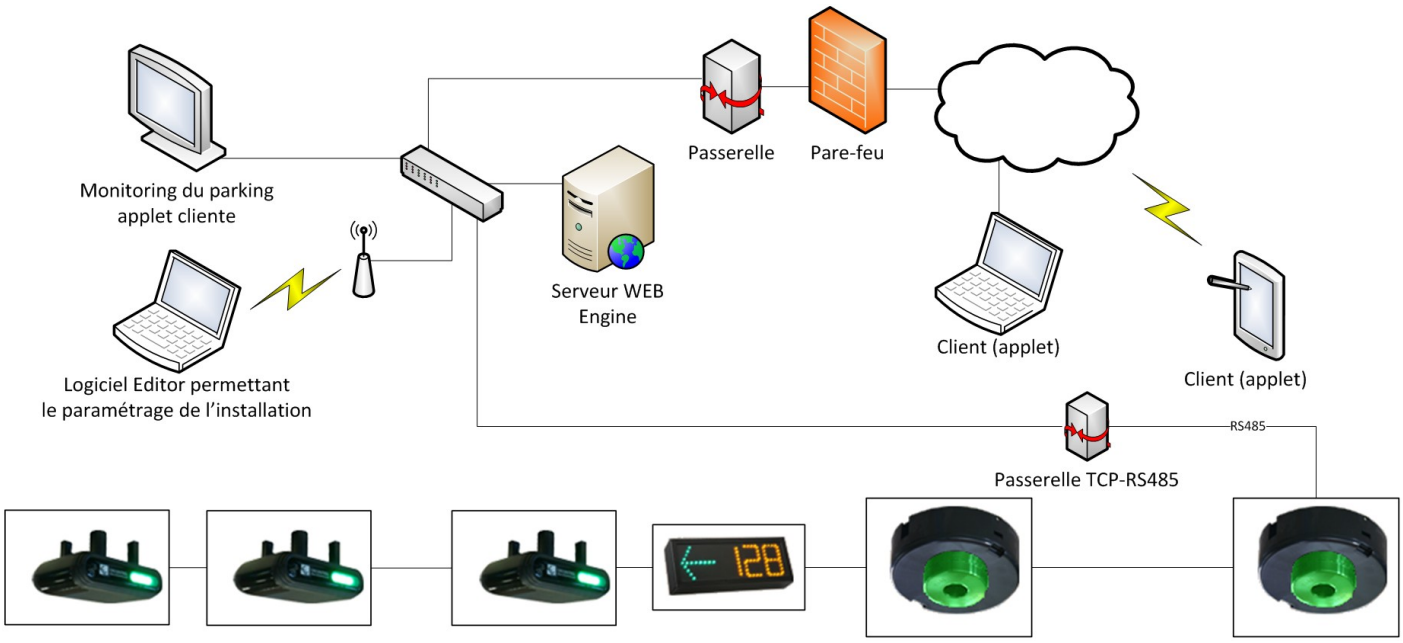
Décrire en quelques mots de format XML :

Quelle est la différence entre le format JSON et le format XML ?

L'installation est construite autour du serveur qui héberge « l'Engine » à savoir le serveur Web. Sur la partie LAN, seront connectés la (ou les) passerelle(s) TCP-RS485 qui permet(tent) la communication entre l'Engine et les divers capteurs, détecteurs et afficheurs installés sur le parking.

Donner les caractéristiques essentielles du bus RS485 :

Le monitoring est assuré par un écran intelligent client de l'Engine. Un ordinateur comportant le logiciel d'édition permet d'éditer le paramétrage de l'installation et de publier la configuration vers l'Engine. Enfin des clients Web (côté WAN du serveur) peuvent lancer une requête afin de connaître l'état du parking (places disponibles).



En étudiant la page 14 de la documentation catalogue-product-cirpark.pdf, donner le principe physique de détection utilisé par un capteur SP3. :

TP

Installation du système Cirpark

Chaque étudiant va installer le système sur son poste de travail, dans un premier temps, les capteurs seront simulés, leur état sera placé dans le fichier texte (vectorCapteurs.vec).

Installation des logiciels constructeur

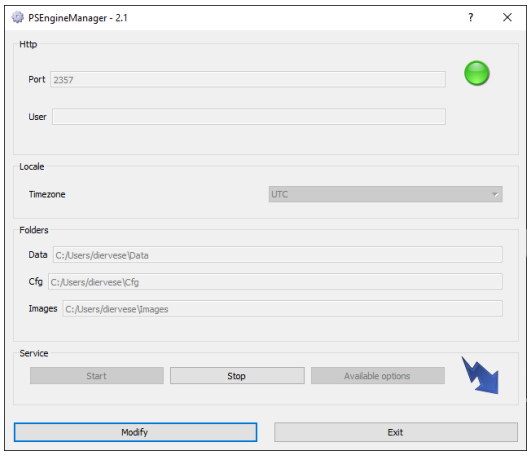
> Installer Cirpark 4.5.3-rc1.exe, continuer même si certains paquets indiquent des erreurs : Compagny name LLA.

Ajout de capteurs simulés dans l'environnement Cirpark

> Lancer SimulateurCapteursServeurUDPCirpark.exe :les capteurs simulés sont dans le fichier vectorCapteurs.vec. Ce simulateur est un serveur UDP.

Lancement du serveur Web contenant la configuration de la passerelle TCP-RS85

> Lancer Cirpark scada engine manager, puis ServeurConfigTCPRSCirpark : il faut modifier l'adresse IP présente dans le fichier de configuration ConfigTCPRSCirparkHTTP.txt. Le serveur Web ServeurConfigTCPRSCirpark transmettra à l'« engine » un fichier JSON contenant sa configuration. NB : l'« engine » devra être redémarrer (STOP puis START) chaque heure.



Configuration de scada editor : ajout graphique de la passerelle TCP-RS485 et des capteurs

> Lancer Cirpark scada editor : ajouter des dispositifs… En haut à gauche : Préférence moteur, placer la bonne adresse, « +de moteurs » doit passer au vert.

Ajout d'un dispositif TCP/IP : TCP3RS

> - Nom TCPsimuRS
> - Adresse IP du PC
> - Ports 10001 et 20002
> - Adresse IP du moteur : adresse du PC
> - Port 80
> - …assigner une adresse ip au dispositif TCP3RS

Ajout des capteurs

Ajouter un nouveau dispositif :

...Dispositifs connectés à un autre dispositif

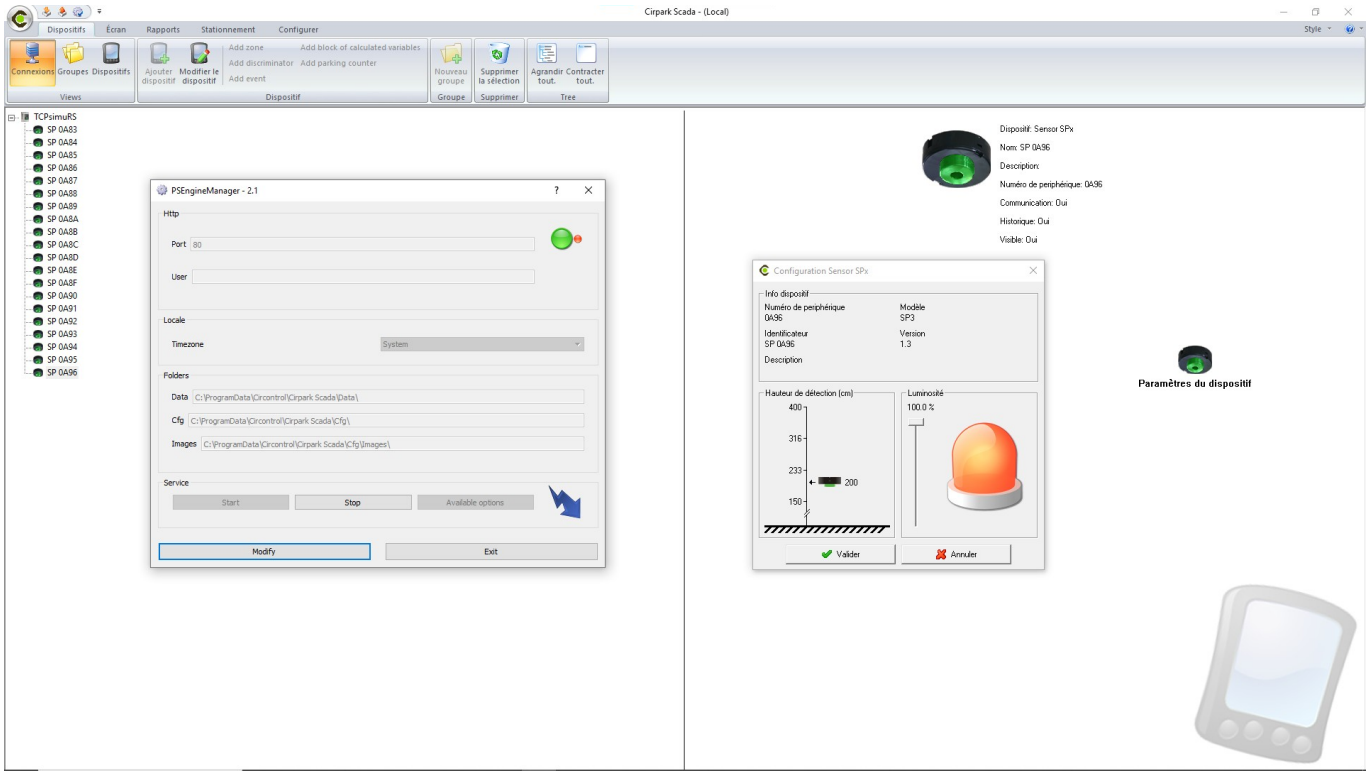
Choisir le TCPsimuRS

...Stationnement

- sensor SPx
- Ajouter plusieurs capteurs : 20
- nom SP
- numéro 0A82

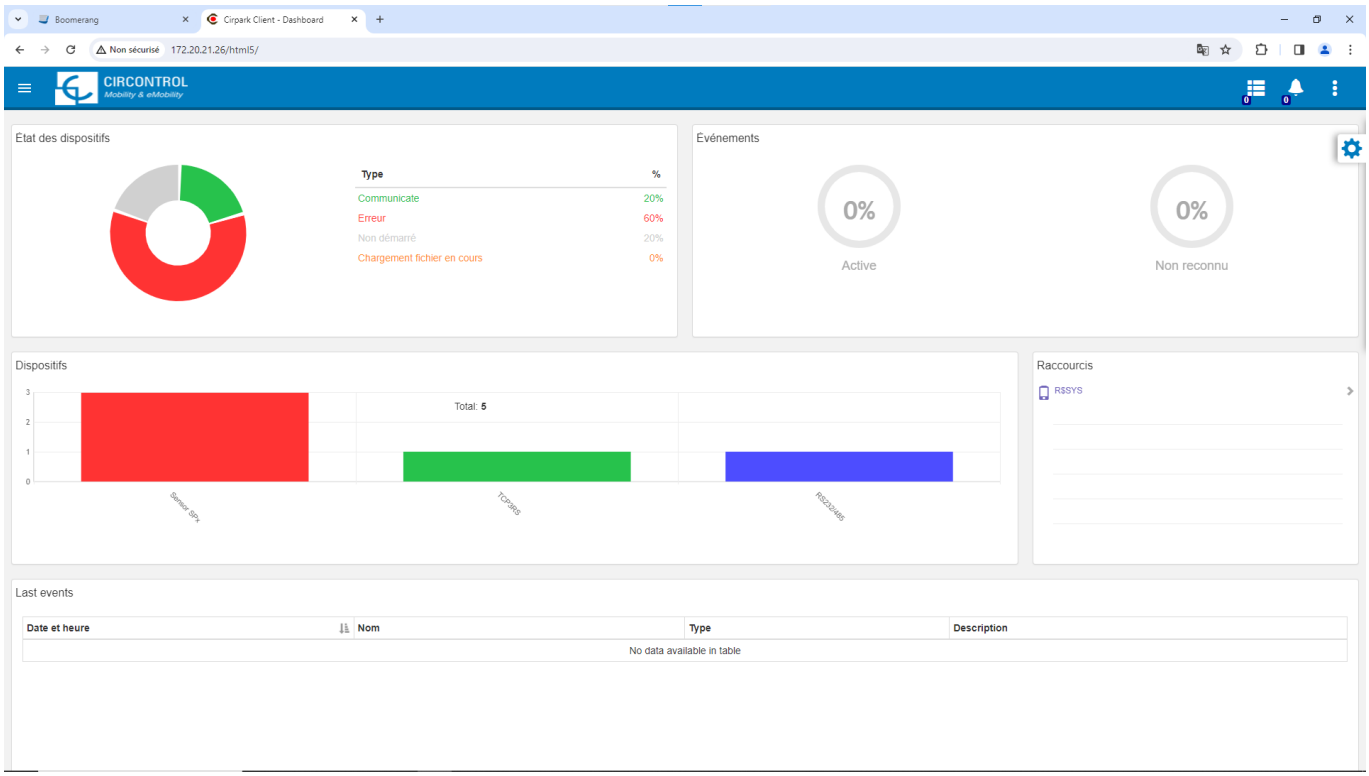
Exporter (F6) vers Engine

Dans Cirpark Scada : choisir un capteur et vérifier les paramètres du dispositif, ils doivent correspondre aux paramètres présents dans vectorCapteurs.vec,



Consultation du site Web de l'« engine »

Vérifier dans un navigateur (http://172.20.21.20/html5/) le nombre de capteurs connectés.



Analyse Wireshark de la communication

Lancer la capture en plaçant les filtres adéquats. Repérer les trames HTTP entre l'« engine » et le serveur Web de configuration du TCP-RS485, entre l'« engine » et le serveur UDP simulant les capteurs. Indiquer les filtres utilisés :

Bonus : ajout d'un capteur réel

Connecter en USB le capteur sur le poste de travail. Dans Cirpark scada editor, ajouter un élément port série auquel il faudra connecter le capteur réel. Tester le fonctionnement, puis analyser les échanges en utilisant serial port monitor.

TP

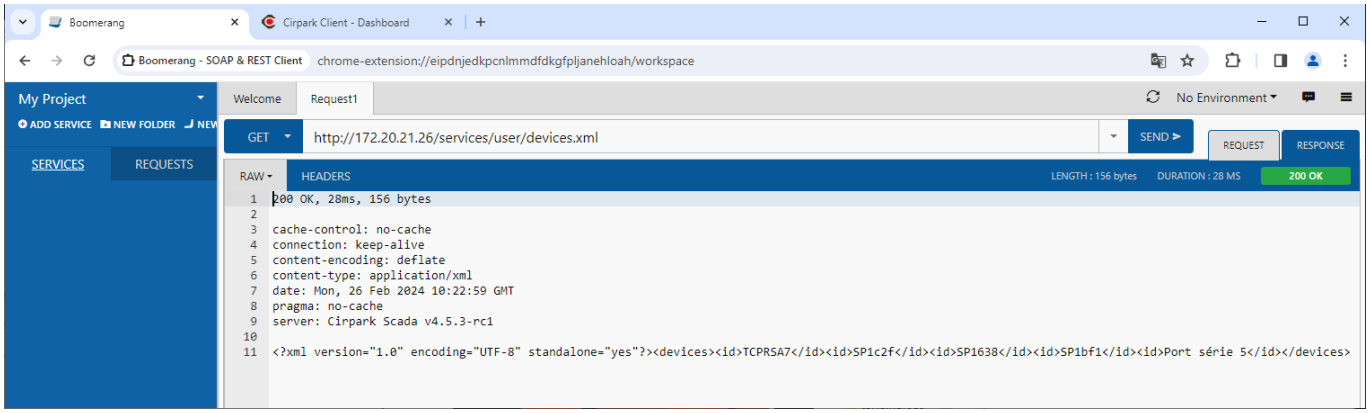
Test de l'API HTTP (xml)

Étudier la documentation : MU610105-16A-EN-Cirpark Scada-5-XML Protocol…
Que permet la requête : /services/user/devices.xml ?

Que permet la requête : /services/user/deviceInfo.xml?id=SP%200A83 ?

Que permet la requête : /services/user/values.xml?var=SP%200A83.STATE ?

Tester les requêtes dans un client REST, puis analyser les réponses XML.



Analyser sous Wireshark les échanges en prenant soin de recopier les requêtes complètes dans un fichier texte : ces requêtes seront ensuite intégrées dans un code C++. Pour chaque requête, noter le nombre de trames TCP constituant la réponse HTTP.

MODULE 06

SÉANCE SYSTÈME 02
TP D'INFORMATIQUE

Durée 2h30

CODAGE DE LA REQUÊTE HTTP EN C++ : EXTRACTION DE LA LISTE DES APPAREILS (EXTRACTION XML)

BLOC DE COMPÉTENCES

U6 - VALORISATION DE LA DONNÉE ET CYBERSÉCURITÉ

COMPÉTENCE(S)

C08 - CODER

OBJECTIFS PÉDAGOGIQUES

Codage d'un client TCP permettant d'envoyer les requêtes GET nécessaires à l'identification des appareils présents dans l'installation.

CONNAISSANCES ISSUES DU RÉFÉRENTIEL

- Programmation orientée objet — Niveau 3
- Programmation réseau — Niveau 3
- Outils logiciels d'évaluation, de traçabilité de l'information — Niveau 3

CONNAISSANCES OPÉRATIONNALISÉES

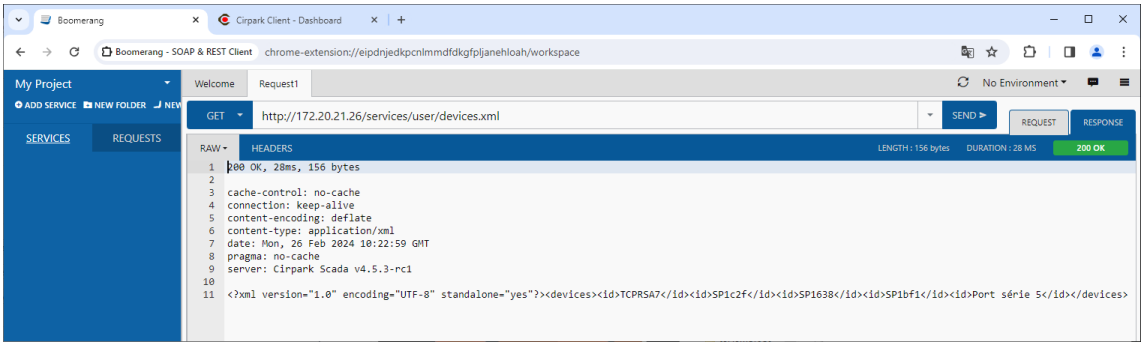
- Envoyer une trame TCP — Niveau 3
- Analyser une communication réseau avec Wireshark — Niveau 3
- Versionner un code — Niveau 3

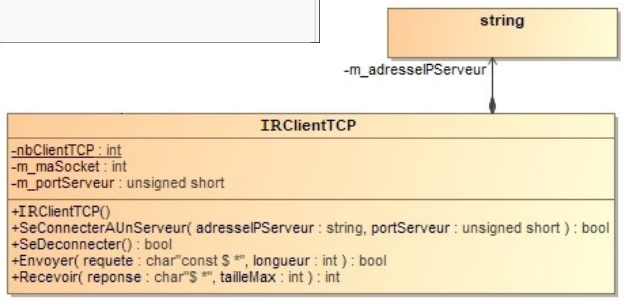
Gaëtan GEORGES
Pascal HAGOT
David IERVESE

Lycée Louis Armand
173 Bd de Strasbourg
Nogent sur Marne

TP

Envoi de la requête GET et affichage de la réponse XML





Créer le programme C++ permettant de se connecter au serveur HTTP (classe IRClientTCP). Envoyer la requête GET identifiée dans la dernière séance (MS06SS01). Votre code :

La réponse est contenue dans 2 trames TCP / IP : appeler deux fois la méthode Recevoir() de la classe IRClientTCP, en concaténant les réponses dans un string. Penser à se déconnecter du serveur.

Afficher la réponse et le contenu XML. Votre code :

Bonus : placer dans un fichier de configuration l'adresse et le port, qu'un objet de la classe ifstream sera chargé de lire. Intégrer ces paramètres dans votre code ainsi qu'à l'intérieur de la requête HTTP à envoyer au serveur.

```
172.20.21.26
80
```

TP Extraction de la liste des appareils présents dans la réponse XML

Rechercher la documentation de la méthode find de la classe string : donner un exemple d'utilisation :

Quelle valeur est retournée lorsque la chaîne de caractères n'est pas trouvée ?

Rechercher la documentation de la méthode substr() de la classe string : donner un exemple d'utilisation :

A l'aide la la méthode find() rechercher dans la réponse xml du serveur la chaine "<id>" en plaçant l'indice retourné dans une variable entière debut. Votre code :

Rechercher ensuite la chaine "</id>" en plaçant l'indice retourné dans une variable entière fin. Votre code :

Afficher le contenu des variables début et fin et vérifier leur valeur en comptant les caractères dans la réponse xml (utiliser Notepad C++ pour vérifier les indices).

Utiliser la méthode substr() de la classe string afin d'afficher le nom du premier appareil : justifier les valeurs des arguments :

```
substr(debut+4,fin-debut-4)
```

Mettre en place une boucle do...while permettant d'extraire le nom de tous les appareils : do... while(debut!=-1).

Lancer une capture Wireshark permettant de visualiser la connexion TCP, l'envoi de la requête, la réception des données XML ainsi que la déconnexion. Quel filtre utiliser ?

Versionner le code complet.

Bonus

> Gérer les requêtes permettant d'avoir les informations d'un appareil (/services/user/deviceInfo.xml?id=SP...), %20 replacera les espaces présents dans le nom de l'appareil.

> Gérer les requêtes permettant d'obtenir l'état d'un capteur (/services/user/values.xml?var=SP... .STATE)

> Versionner le code complet.

MODULE 06

SÉANCE SYSTÈME 03

TP D'INFORMATIQUE

Durée 2h30

La classe IRSnifferCirpark : détection d'un capteur

BLOC DE COMPÉTENCES

U6 - VALORISATION DE LA DONNÉE ET CYBERSÉCURITÉ

COMPÉTENCE(S)

C08 - CODER

OBJECTIF PÉDAGOGIQUE

Dans cette partie, vous allez créer une nouvelle classe, envoyer des trames sur le réseau et recevoir des réponses et stocker les résultats dans un tableau de données.

CONNAISSANCES ISSUES DU RÉFÉRENTIEL

- Programmation Orientée Objet — Niveau 3

CONNAISSANCES OPÉRATIONNALISÉES

- Utiliser une classe — Niveau 2
- Créer une classe — Niveau 2

Gaëtan GEORGES
Pascal HAGOT
David IERVESE

Lycée Louis Armand
173 Bd de Strasbourg
Nogent sur Marne

Cahier des charges et analyse

Cahier des charges

L'objectif de cette partie est de mettre en place un sniffer Cirpark. Un sniffer est un outil qui permet de découvrir un réseau et de trouver par lui-même tous les éléments qui le compose.

Notre sniffer Cirpark aura pour mission de découvrir tous les capteurs présents dans le parking. Pour cela, il interrogera tous les capteurs possible et attendra une réponse. Si le capteur répond, il l'ajoutera à la liste des capteurs trouvés.

Analyse

> Expliquer le but de notre sniffer Cirpark.

> Donner le nombre d'octets d'une adresse de capteurs SP.

> En déduire par un calcul le nombre d'adresse possible pour les capteurs SP.

> Supposons que la découverte d'un seul capteur prenne 200ms. Calculer le temps nécessaire pour les découvrir tous.

> En déduire l'intérêt de faire une chercher des capteurs par plage d'adresse (une plage d'adresse est définie par une adresse de départ et une adresse de fin)

Architecture matérielle

Votre sniffer Cirpark ne communiquera par directement sur le réseau RS485 des capteurs. Il passera par un TCPRS qui sert d'interface entre le réseau RS485 des capteurs et le réseau Ethernet. Le TCPRS possède un serveur UDP qui reçoit des trames provenant du réseau Ethernet. Ces trames contiennent un message pour un capteur. Le TCPRS transmet ce message sur le réseau RS485 des capteurs. Si le capteur existe et que le message est correct, le capteur répond. Ensuite, le TCPRS retransmet la réponse du capteur au client UDP.

Dessiner le diagramme de déploiement en indiquant les 3 éléments présents, leur multiplicité et les protocoles qui les relient.

Le protocole de communication

D'après la documentation du protocole des capteurs de type SP, quelle trame permet de connaître le type de capteur ? Quels sont les différents champs de cette trame ?

Comment se calcule le champ bcc ?

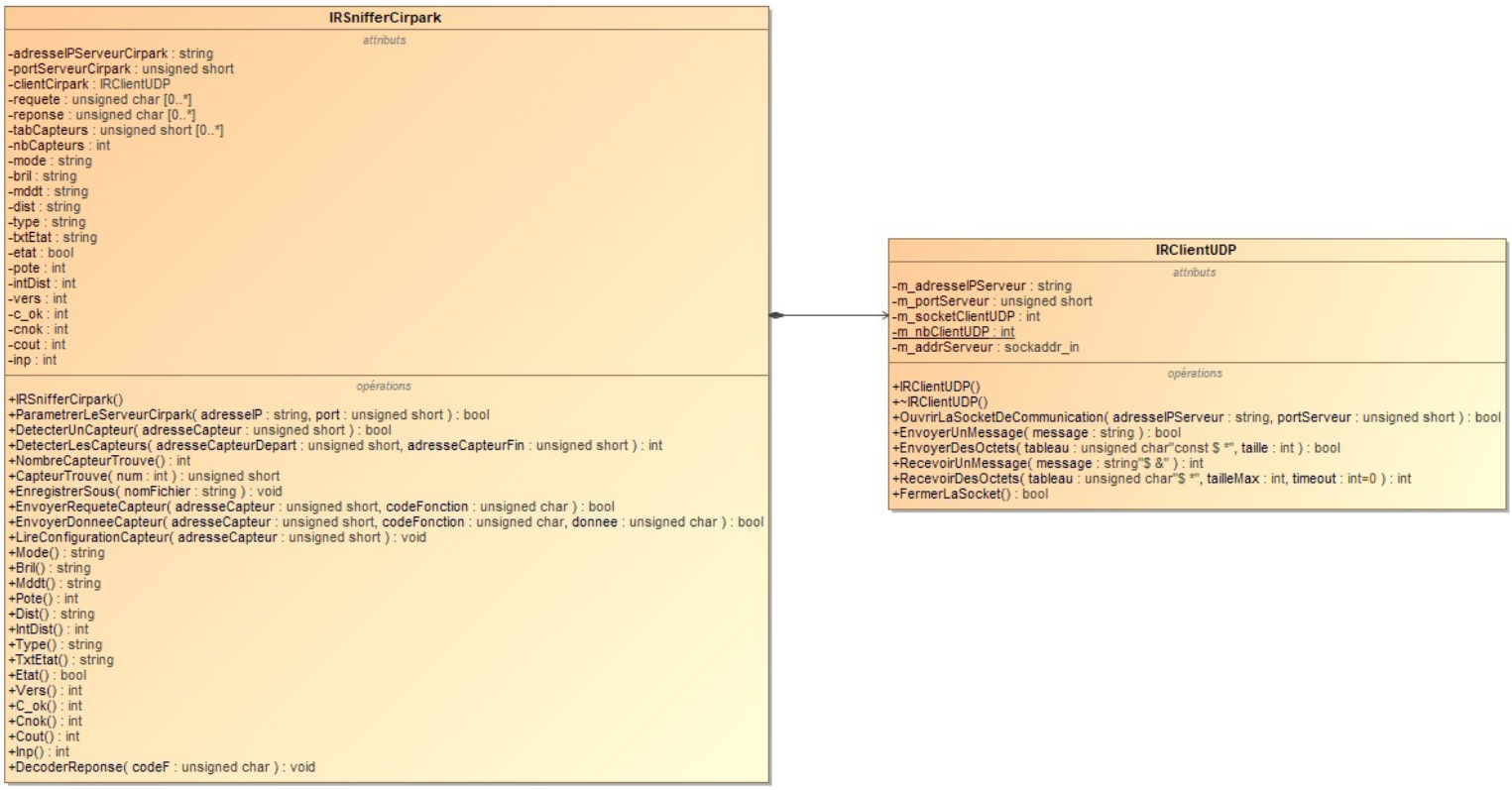
Donner la trame UDP à envoyer au capteur 1E3F pour connaître son type. Calculer le bcc.

Ce sont ces trames là que nous allons envoyer pour découvrir les éléments présents sur le réseau Cirpark. Si nous obtenons une réponse, cela signifie que le capteur existe. Si nous n'avons pas de réponse, on en déduit que le capteur n'est pas présent.

Ecriture de la classe IRSnifferCirpark

Le diagramme de classe

Soit le diagramme de classe complet suivant :



La classe IRClientUDP vous est fournie.

> Donner le nom du lien qui relie les classes IRSnifferCirpark et IRClientUDP.

> Quelle est la conséquence de ce lien pour la classe IRSnifferCirpark ? Surligner la ligne correspondante.

Nous allons écrire la classe IRClientCirpark progressivement et la tester progressivement également.

Création d'un nouveau projet avec XE6

> Avec XE6, créer un projet en mode console. Enregistrer votre projet sous... dans un répertoire dédié.
> Ajouter dans votre répertoire de projet les sources de la classe IRClientUDP. Puis ajouter dans votre projet ces fichiers.

Ajout des fichiers pour la classe IRSnifferCirpark

Dans votre projet, créer une nouvelle Unité. Ceci à pour conséquence de créer 2 fichiers (.h et .cpp) que vous utiliserez pour créer la classe IRSnifferCirpark. Sauvegarder le fichier Unit1.h sous le nom « IRSnifferCirpark.h ». Le fichier Unit1.cpp est alors automatiquement renommé.

Déclaration partielle de la classe IRSnifferCirpark

Dans le fichier IRSnifferCirpark.h, déclarer la classe IRSnifferCirpark. On commencera avec seulement les attributs et méthodes suivantes :

- Les attributs : adresseIPServeurCirpark, portServeurCirpark, clientCirpark, requete (un tableau de 10 octets non signés), reponse (un tableau de 20 octets non signés), tabCapteurs (un tableau de 100 adresse de capteurs sur 16 bits), nbCapteurs.
- Les méthodes : Le constructeur, ParametrerLeServeurCirpark(), DetecterUnCapteur(), DetecterLesCapteurs(), NombreCapteurTrouve(), CapteurTrouve().

Définition des méthodes

Le constructeur

Le constructeur initialise les attributs de la classe.

Dans la définition du constructeur, attribuer la valeur « 0.0.0.0 » à adresseIPServeurCirpark, 10001 à portServeurCirpark, mettre des 0 dans toutes les cases des tableaux requete, reponse et tabCapteurs (vous pouvez faire une boucle for pour initialiser à 0 tous les éléments d'un tableau ou bien utiliser la fonction memset) et enfin 0 dans nbCapteurs.

Vérifier qu'il n'y pas d'erreur de compilation.

La méthode ParametrerLeServeurCirpark()

Cette méthode permet à l'utilisateur d'indiquer l'adresse IP et le port du module TCPRS avec lequel on souhaite communiquer. Cette méthode créer également la socket de communication.

On vous donne ci-dessous le code partiel de cette méthode :

```
1: bool IRSnifferCirpark::ParametrerLeServeurCirpark(string adresseIP, unsigned short port)
2: {
3:   adresseIPServeurCirpark = ________________;
4:   portServeurCirpark = _______________;
5:   ______  _________ = clientCirpark.OuvrirLaSocketDeCommunication(adresseIPServeurCirpark, portServeurCirpark);
6:   return ___________;
7: }
```

Compléter le code ci-dessus en vous aidant des indications qui suivent :

ligne 3 : l'attribut adresseIPServeurCirpark prend la valeur du 1er argument de la méthode (adresseIP).

ligne 4 : l'attribut portServeurCirpark prend la valeur du 2ème argument de la méthode (port).

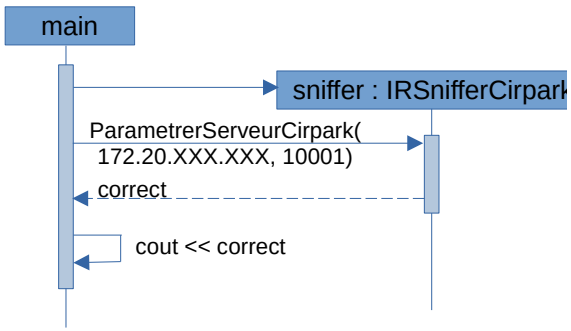
Ligne 5 : La valeur de retour de la méthode OuvrirLaSocketDeCommunication() est affectée à un booléen nommé « *retour* » qu'il faut créer.

Ligne 6 : La méthode renvoie la valeur du booléen « *retour* ».

Test du constructeur et de la méthode ParametrerLeServeurCirpark()

A partir du diagramme de séquence ci-contre, écrire le programme principal main.

Lors de l'exécution, vérifier que votre programme affiche « 1 ». Ceci indique que la méthode ParametrerServeurCirpark() a correctement fonctionné.



La méthode DetecterUnCapteur()

Maintenant que la socket de communication est créée avec la méthode ParametrerServeurCirpark(), il est temps d'envoyer une trame au serveur pour découvrir si un capteur est présent ou pas. Nous arrivons ici au coeur de notre projet et c'est le but de la méthode DetecterUnCapteur().

Pour savoir si un capteur est présent, nous allons utiliser la fonction 0x05.

D'après la documentation, de combien d'octet est composée la requête à envoyer aux capteurs SP3 ? Donner leur signification.

En supposant que le capteur est 0x1EB9, donner la trame complète qu'il faut envoyer.

adrh	adrl	fonction	bcc

Le capteur qui recevra cette requête renverra son type de capteur s'il est présent.

Donner le nom de l'argument de la méthode DetecterUnCapteur() et son type.

Décomposons maintenant les instructions de la méthode DetecterUnCapteur().

En C++, donner l'opération permettant d'extraire l'octet de poids fort de l'adresse du capteur passé en argument dans le 1er octet de la requête :

requete[0] =

En C++, donner l'opération permettant d'extraire le l'octet de poids faible de l'adresse du capteur passé en argument dans le second octet de la requête :

requete[1] =

En C++, compléter maintenant le troisième octet avec la valeur de la fonction :

requete[2] =

> Pour terminer, compléter le quatrième octet avec la valeur du bcc (la somme algébrique des précédents octets :

requete[3] =

> En C++, appeler la méthode EnvoyerDesOctets() avec l'objet clientCirpark. Passer en argument le tableau « requete » ainsi que le nombre d'octets à envoyer.

> En C++, appeler la méthode RecevoirDesOctets(). Passer en argument le tableau « reponse » avec sa taille max de 20 octets. Stocker la valeur renvoyée dans un entier « nbOctets » que vous déclarerez.

> Terminons la méthode. En C++, retourner « false » si nbOctets vaut 0 ou « true » sinon.

> Écrire la définition complète de la méthode DetecterUnCapteur() dans le fichier IRSnifferCirpark.cpp.

Testons maintenant cette méthode.

> Dans le main(), appeler cette méthode avec l'objet « sniffer » et afficher la valeur que cette méthode renvoie. Pour que votre test soit complet, passer en argument un capteur qui existe puis un capteur qui n'existe pas. Vérifier avec Wireshark que la trame de requête est correcte et que vous obtenez une trame de réponse lorsque le capteur existe.

Arrivé à ce stade, nous avons écrit un programme qui est capable de savoir si un capteur est présent ou non. Passons maintenant à l'étape suivante : créer une méthode qui vérifie automatiquement si des capteurs sont présents dans une plage d'adresse.

MODULE 06

SÉANCE SYSTÈME 04

TP D'INFORMATIQUE

Durée 2h30

La classe IRSnifferCirpark – détection des capteurs

BLOC DE COMPÉTENCES

U6 - VALORISATION DE LA DONNÉE ET CYBERSÉCURITÉ

COMPÉTENCE(S)

C08 - CODER

OBJECTIF PÉDAGOGIQUE

Dans cette partie, vous travaillerez sur des tableaux de données. Il faudra parcourir des tableaux, les afficher selon un format prédéfini et les enregistrer les données qu'ils contiennent dans un fichier.

CONNAISSANCES ISSUES DU RÉFÉRENTIEL

- Programmation Orientée Objet — Niveau 3

CONNAISSANCES OPÉRATIONNALISÉES

- Utiliser une classe — Niveau 2
- Créer une classe — Niveau 2

Gaëtan GEORGES
Pascal HAGOT
David IERVESE

Lycée Louis Armand
173 Bd de Strasbourg
Nogent sur Marne

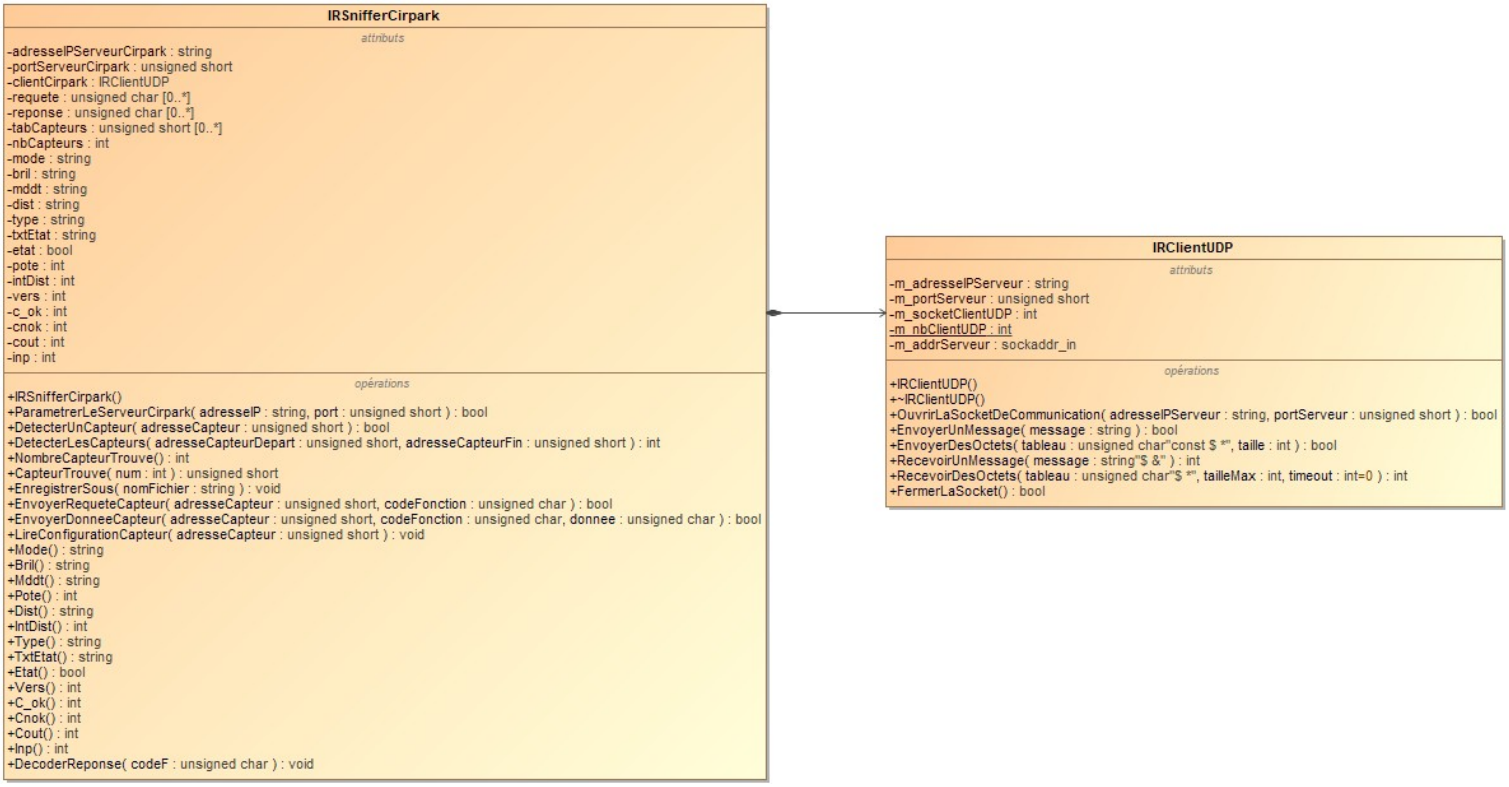
Écriture de la classe IRSnifferCirpark

Présentation

Dans cette deuxième partie, nous allons travailler sur la méthode qui permet de connaître automatiquement la liste des capteurs présents sur une plage d'adresse : c'est la méthode DetecterLesCapteurs() de la classe IRSnifferCirpark.

Le diagramme de classe

Pour rappel, voici le diagramme de classe complet :



La méthode DetecterLesCapteurs()

La définition de la méthode DetecterLesCapteurs()

Cette méthode permet à l'utilisateur de connaître la liste des capteurs présents dans un intervalle d'adresse. Par exemple, si nous souhaitons connaître le nombre de capteur entre les adresses 0x0000 et 0x1FFF, on écrira :

```
sniffer.DetecterLesCapteurs(0x0000, 0x1FFF);
```

Donner le nombre d'argument attendu par la méthode DetecterLesCapteurs() et leur type. Justifier le type choisi.

Donner le type de retour de cette méthode.

> Cette méthode renvoie l'attribut nbCapteurs (qui contient le nombre de capteurs trouvés). Est-ce compatible avec le type de retour de la méthode ? Justifier.

> Au début de cette méthode, vous initialiserez nbCapteurs à 0 et vous remplirez le tableau tabCapteurs de 0. Ecrire le code correspondant.

Pour détecter combien de capteurs sont présents dans une plage d'adresse, nous allons nous aider de la méthode DetecterUnCapteur() qui nous indique si un capteur est présent... Et nous allons répéter du premier capteur demandé au dernier capteur demandé.

> A votre avis, quelle boucle est préférable : for, while ou do ... while ? Justifier ?

> De quel type sera la variable de boucle ? Justifier. Vous la nommerez « adr ».

Test de la méthode DetecterLesCapteurs()

Dans le programme principal, appeler la méthode DetecterLesCapteurs() dans la plage 0x01a0 à 0x01ff. Puis afficher la valeur retournée par cette méthode.

> Ecrire le code C++ correspondant.
>
> Vérifier que cela correspond bien avec la réalité. Lancer Wireshark pour vérifier que vos trames émises sont correctes.

La méthode NombreCapteurTrouve()

Cette méthode est une méthode d'accès qui renvoie l'attribut nbCapteur.

> Ecrire sa définition en C++ et la tester dans votre programme principal.

La méthode CapteurTrouve()

Cette méthode est une méthode d'accès au tableau tabCapteurs. Elle renvoie le capteur qui se trouve à l'indice num passé en argument de cette méthode.

> Ecrire la définition de la méthode unsigned short CapteurTrouve(int num). Si l'argument num est inférieur à nbCapteur, cette méthode renvoie le capteur se trouvant à l'indice num dans le tablea tabCapteurs. Sinon, cette méthode renvoie 0.
>
> Tester cette méthode dans votre programme principal en affichant le premier capteur que votre sniffer a trouvé. Vous veillerez à afficher le numéro du capteur sur 4 chiffres en hexadécimal et en majuscule (en complétant avec des '0' devant si nécessaire). Voici comment réaliser un tel affichage :

```
unsigned short capteur = 0x01ec;
cout << uppercase << hex << setw(4) << setfill('0') << capteur << std::endl;
```

Pour utiliser les fonctions setw() et setfill(), il faut inclure le fichier :

```
#include <iomanip>
```

> Tester cette méthode dans votre programme principal en affichant tous les capteurs que votre sniffer a trouvé.

La méthode EnregistrerSous()

L'objectif de cette méthode est d'enregistrer dans un fichier tous les capteurs qui ont été trouvé. Le nom du fichier est passé en argument. Tous les capteurs seront enregistrés sur 4 chiffres en hexadécimal et en majuscule (en complétant avec des '0' devant si nécessaire).

> Rappeler le nom de la classe permettant d'écrire dans un fichier.

```
```

> Dans la définition de la méthode EnregistrerSous() :
> - créer un objet de la classe ofstream
> - ouvrir le fichier passer en argument
> - faire une boucle sur l'ensemble des capteurs trouvés et ajouter à chaque fois une nouvelle ligne dans le fichier en respectant le format imposé.

Tester la méthode EnregistrerSous() dans votre programme principal. Vérifier qu'un fichier est bien créé et qu'il contient bien la liste de tous les capteurs trouvés.

MODULE 06

SÉANCE SYSTÈME 05

TP D'INFORMATIQUE

Durée 2h30

Le TCP RS sur Raspberry

BLOC DE COMPÉTENCES

U6 - VALORISATION DE LA DONNÉE ET CYBERSÉCURITÉ

COMPÉTENCE(S)

C08 - CODER

OBJECTIF PÉDAGOGIQUE

Dans cette partie, vous utiliserez une classe documentée pour créer un serveur UDP sur une Raspberry.

CONNAISSANCES ISSUES DU RÉFÉRENTIEL

- Programmation Orientée Objet — Niveau 3

CONNAISSANCES OPÉRATIONNALISÉES

- Utiliser une classe — Niveau 2
- Créer une classe — Niveau 2

Gaëtan GEORGES
Pascal HAGOT
David IERVESE

Lycée Louis Armand
173 Bd de Strasbourg
Nogent sur Marne

Le TCP RS sur Raspberry

Présentation de l'évolution du système

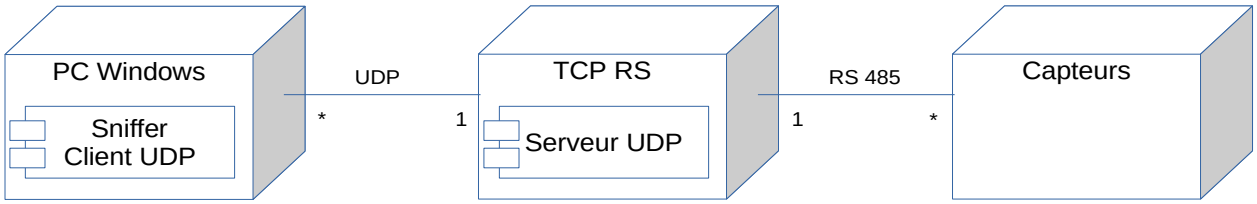
Dans les parties qui vont suivre, vous allez effectuer une mutation de ce projet. En effet, nous souhaitons changer les TCP RS par des Raspberry. Voici au moins 2 raisons :

1. Les TCP RS sont très vite saturés si plusieurs clients font des requêtes simultanément.
2. Les protocoles des TCP RS ne sont pas chiffrés.

Votre mission consistera donc à améliorer l'existant : en utilisant un système plus performant avec chiffrement.

Le diagramme de déploiement

Pour rappel, voici le diagramme de déploiement actuel.



Le TCP RS possède donc un serveur UDP qui reçoit toutes les requêtes des clients UDP (de votre application sniffer par exemple). Lorsqu'il reçoit une requête, le TCP RS la renvoie telle quelle sur le bus RS 485. Les capteurs voient ainsi les requêtes. Si l'un d'eux est concerné (c'est son adresse sur les 2 premiers octets), alors il y répond sur le bus RS 485. Le TCP RS voit la réponse et la transmet au client.

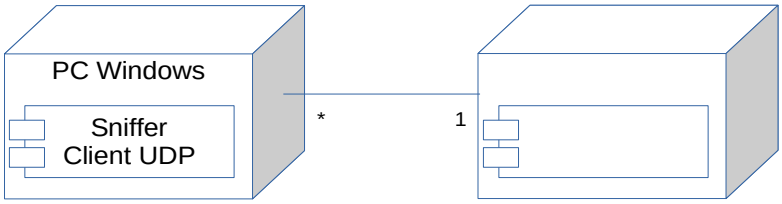
Les phases de l'évolution du système :

- 1ère phase : Mettre en place un serveur UDP sur une Raspberry
- 2ème phase : Chiffrer les données entre le client et le serveur UDP
- 3ème phase : Communiquer avec les capteurs sur le bus RS 485

1ère phase : Mettre en place un serveur UDP sur Raspberry

Dans cette première évolution du système, vous allez mettre en place un serveur UDP sur Raspberry. Il répondra systématiquement à toutes les requêtes de fonction 0x05 provenant des clients.

Compléter le diagramme de déploiement de la 1ère phase en ajoutant les mots : Rapsberry, UDP, Serveur UDP aux bons endroits :



Mise en place d'un projet C++ sur la Raspberry

Sur le répertoire d'échange, récupérer le dossier RaspberryServeurUDP_PourEtudiants

Avec SSHFS-Win-Manager (ou WinSCP), se connecter sur la Raspberry cible avec vos identifiants et coller le dossier que vous venez de récupérer.

Ouvrir les fichiers sources du projet avec VS Code ou un autre éditeur.

Modifier le programme principal pour qu'il affiche « Serveur UDP sur Raspberry ! »

Rappeler la commande nécessaire pour lancer une compilation et le fichier indispensable contenant les commandes de compilation.

Ouvrir le fichier makefile et trouver le nom de l'exécutable qui sera générer lors de la compilation

Compiler et exécuter le programme. Vérifier que le message apparaît.

Création d'un serveur UDP en C++

Pour vous faciliter la mise en place du serveur UDP en C++, nous avons mis au point une classe : IRServeurUDP qui est déjà présente dans le répertoire du projet que vous avez téléchargé. Voici son diagramme UML de classe (Une documentation de cette classe sera accessible ici : http://172.20.21.200/IRClasses)

IRServeurUDP
- m_adresseIPDEcoute : string - m_portDEcoute : unsigned short - m_socketServeurUDP : int - m_nbServeurUDP : int - m_addrServeur : struct sockaddr_in - from : struct sockaddr_in
+ IRServeurUDP() + ~IRServeurUDP() + OuvrirLaSocketDEcoute(portDEcoute : unsigned short, adresseIPDEcoute : string) : bool + EnvoyerUnMessage(message : string) : bool + EnvoyerDesOctets(tableau : const unsigned char *, taille : int) : bool + RecevoirUnMessage(message : string &) : int + RecevoirDesOctets(tableau : unsigned char *, tailleMax : int) : int + FermerLaSocket() : bool

Première version : un serveur UDP qui affiche le message reçu

Le but de cette première version est de faire un serveur UDP qui reçoit un seul message et le renvoie comme un écho. Ensuite, le serveur s'arrête.

> En C++, créer un objet « serveur » de la classe IRServeurUDP.

> En C++, ouvrir la socket d'écoute du serveur sur toutes les adresses IP disponibles de la Raspberry et sur le port 10100 + numéro de poste.
>
> Expliquer pourquoi vous ne pouvez pas tous avoir le même port d'écoute sur la Raspberry ?

> En C++, ajouter les instructions permettant de récupérer les octets envoyés par le client (votre sniffer) avec la méthode RecevoirDesOctets(). Voir la documentation détaillée de la classe pour vous aider.
>
> En C++, afficher les octets reçus en hexa.
>
> En C++, fermer la socket d'écoute.
>
> Tester votre serveur avec le sniffer des séances précédentes. Avec Wireshark, visualiser les trames circulant sur le réseau entre le client UDP et le serveur UDP.

Deuxième version : un serveur UDP qui répond pour un capteur seulement

Votre serveur n'enverra une réponse qu'à la requête du capteur dont l'id est 0x0821 et si la fonction utilisée vaut 0x05.

> Écrire en C++ cette condition

La réponse devra contenir 5 octets :

- Le premier octet : l'octet de poids fort de l'adresse du capteur
- Le deuxième octet : l'octet de poids faible de l'adresse du capteur
- Le troisième octet : le type de capteur (pour nous ce sont des SP3, voir la documentation)
- Le quatrième : la version (vous indiquerez la version 2.3, voir la documentation)
- Le cinquième octet : ~bcc = ~(octet1 + octet2 + octet3 + octet4)
- Écrire en C++ les instructions permettant de créer un tableau « reponse » contenant les 5 octets précédents.

> En C++, ajouter les instructions permettant de renvoyer les octets reçus. Voir la documentation pour vous aider.
>
> Enfin, en C++, fermer la socket serveur.

Tester votre programme avec votre sniffer et vérifier qu'une réponse est envoyée pour le capteur 0x0821. Avec Wireshark, visualiser les trames circulant sur le réseau entre le client UDP et le serveur UDP.

Troisième version : la boucle infinie

Ajouter une boucle infinie pour que votre serveur puisse lire les messages et y répondre sans jamais s'arrêter.

Tester votre programme.

MODULE 06

SÉANCE SYSTÈME 06

TP D'INFORMATIQUE

Durée 2h30

Chiffrement de la communication

BLOC DE COMPÉTENCES

U6 - VALORISATION DE LA DONNÉE ET CYBERSÉCURITÉ

COMPÉTENCE(S)

C08 - CODER

OBJECTIF PÉDAGOGIQUE

Dans cette partie, vous utiliserez une classe documentée pour créer un serveur UDP sur une Raspberry.

CONNAISSANCES ISSUES DU RÉFÉRENTIEL

- Programmation Orientée Objet — Niveau 3

CONNAISSANCES OPÉRATIONNALISÉES

- Utiliser une classe — Niveau 2
- Créer une classe — Niveau 2

Gaëtan GEORGES
Pascal HAGOT
David IERVESE

Lycée Louis Armand
173 Bd de Strasbourg
Nogent sur Marne

Le chiffrement de données

Présentation

Dans le cadre de la protection des données, nous souhaitons mettre en place un chiffrement des données qui circulent sur le réseau entre le « sniffer » (le client) et le serveur TCP RS sur la Raspberry.

Chiffrement symétrique

> Expliquer ce qu'est un chiffrement symétrique.

> Donner les contraintes de ce type de chiffrement : qui doit connaître la clé, comment la partager en toute sécurité…

Le chiffrement dans le système Cirpark

Dans ce projet, nous utiliserons un chiffrement avec un ou exclusif et une clé de chiffrement sur 2 octets : 0xA3 et 0xC5. Supposons qu'on souhaite envoyer la trame suivante : 0x08 0x12 0x05 0x1F (capteur 0x0812, fonction 05 et bcc).

> Donner la valeur des octets de la trame chiffrée :

1er octet =

2ème octet =

3ème octet =

4ème octet =

> Rappeler le principe du déchiffrement et procéder au déchiffrement de la trame.

Le chiffrement en C++

> Ecrire en C++ la déclaration de la clé de chiffrement sous forme de tableau de 2 octets non signés. Le tableau s'appellera « cle ».

Supposons que nous devions chiffrer le tableau d'octets « requete » suivant :

```
unsigned char requete[4] = {0x08, 0x12, 0x05, 0x1F};
```

dans le tableau « requete_chiffree » déclaré comme suit :

```
unsigned char requete_chiffree[4];
```

> Ecrire les 4 instructions en C++ qui permettront de chiffrer les 4 octets avec la clé précédente (sans faire de boucle).

```
requete_chiffree[0] =
```

> On souhaite optimiser ce code avec une boucle for. Proposer une solution :

Le déchiffrement

> Que dire du déchiffrement ? Comment d'écrit-il en C++ ?

Mise en place du chiffrement dans les applications et test.

Sur le serveur :

Ajouter le déchiffrement des données reçues et le chiffrement des données envoyées.

Sur le sniffer :

Ajouter le chiffrement des données envoyées et le déchiffrement des données reçues.

Vérifier que les échanges se passent correctement. Avec Wireshark, justifier que vos données sont bien chiffrées.

MODULE 06

SÉANCE SYSTÈME 07

TP D'INFORMATIQUE

Durée 2h30

INTERFACE GRAPHIQUE : DÉTECTION AUTOMATIQUE DES CAPTEURS ULTRASON ET AFFICHAGE DES PARAMÈTRES

BLOC DE COMPÉTENCES

U6 - VALORISATION DE LA DONNÉE ET CYBERSÉCURITÉ

COMPÉTENCE(S)

C08 - CODER

OBJECTIFS PÉDAGOGIQUES

Codage d'une interface graphique permettant d'interroger une plage de capteur afin de dresser la liste des appareils connectés. Supervision d'un capteur : lecture et écriture des paramètres.

CONNAISSANCES ISSUES DU RÉFÉRENTIEL

- Langages de développement, de description, de création d'API et les IDE associés — Niveau 3
- Programmation orientée objet — Niveau 3

CONNAISSANCES OPÉRATIONNALISÉES

- Concevoir une interface graphique sous Windows à partir de la documentation d'un matériel. — Niveau 2
- Versionner un code — Niveau 2

Gaëtan GEORGES
Pascal HAGOT
David IERVESE

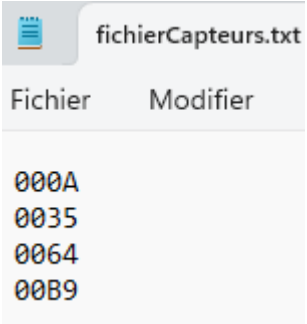
Lycée Louis Armand
173 Bd de Strasbourg
Nogent sur Marne

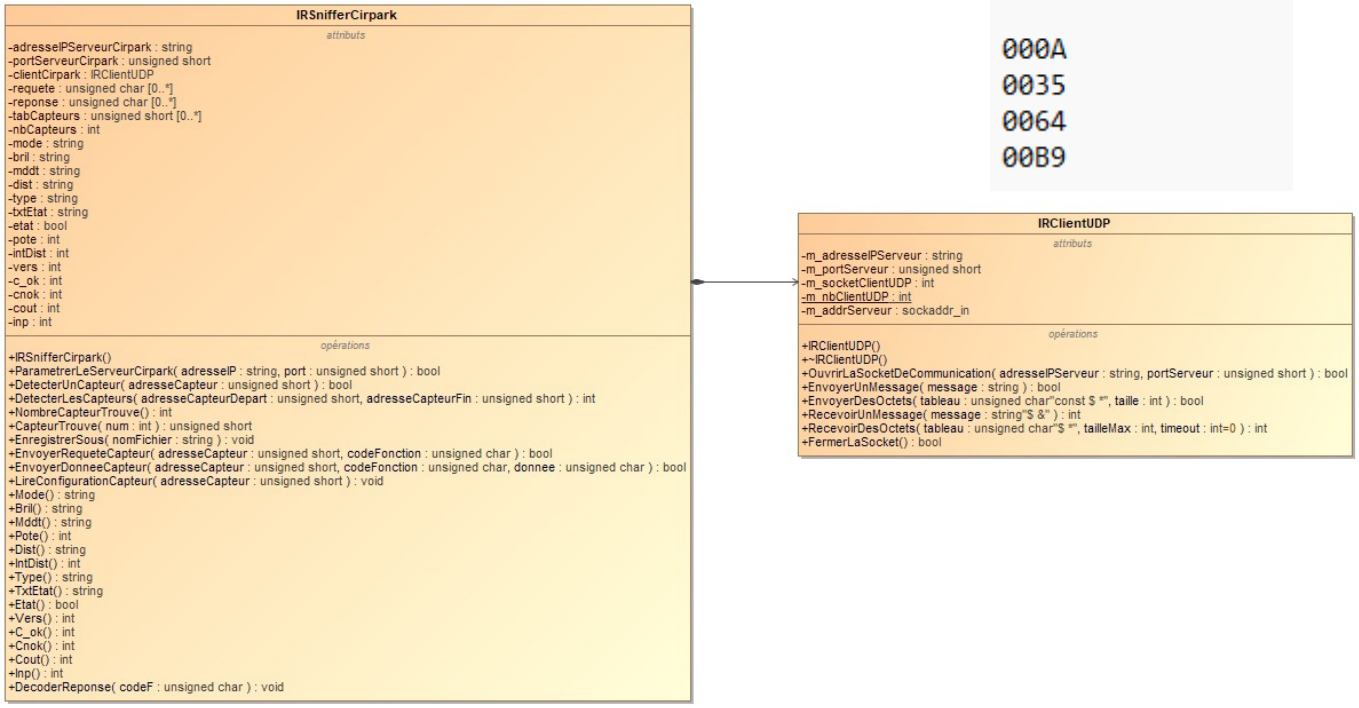
TP
Intégration dans une interface graphique de la classe IRSnifferCirpark : recherche des capteurs

Créer sous C++ Builder, une nouvelle application VCL C++. Ajouter au projet les fichiers nécessaires au fonctionnement de la classe IRSnifferCirpark. Dans la fenêtre de conception, ajouter une zone Edit (qui contiendra l'IP du serveur UDP), un Label et un Button. Dans la classe TForm1 (unit1.h), ajouter, en attribut un objet sniffer de la classe IRSnifferCirpark. Générer l'événement Button1Click. Dans cet événement (unit1.cpp), appeler successivement les méthodes de la classe IRSnifferCirpark :

- ParametrerLeServeurCirpark : en paramètre AnsiString(Edit1->Text).c_str() et 10001 (le port)
- DetecterLesCapteurs : en paramètre la plage de recherche indiquée par l'enseignant
- NombreCapteursTrouves : dont le résultat sera placé dans le Caption du Label
- EnregistrerSous : en paramètre le nom du fichier texte au choix qui contiendra la liste des cateurs

Tester le fonctionnement en vérifiant la liste et le nombre de capteurs.





TP
Lecture de la configuration d'un capteur

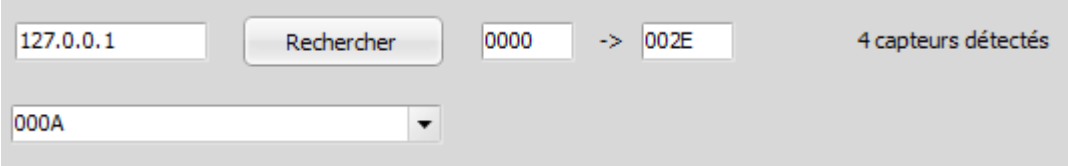
Lecture du fichier texte contenant la liste des capteur, et placement des adresses dans un ComboBox

Toujours dans l'événement Button1Click, Créer un objet de la classe ifstream afin de lire le fichier texte, appeler la méthode open avec cet objet, puis placer chaque adresse de capteur dans le ComboBox avant de refermer la fichier (méthode close()) : on donne :

```
char adresseC[5];
for(int i=0;i<sniffer.NombreCapteursTrouves();i++)
{     f>>adresseC;
      ComboBox1->AddItem(adresseC,this);
}
ComboBox1->Text=adresseC;
```

Bonus : paramétrage de la plage de recherche

Ajouter 2 zones Edit afin de définir la plage de recherche dans l'interface. Utiliser la méthode ChaineHexaVersInt() pour transmettre les 2 valeurs à la méthode DetecterLesCapteurs().

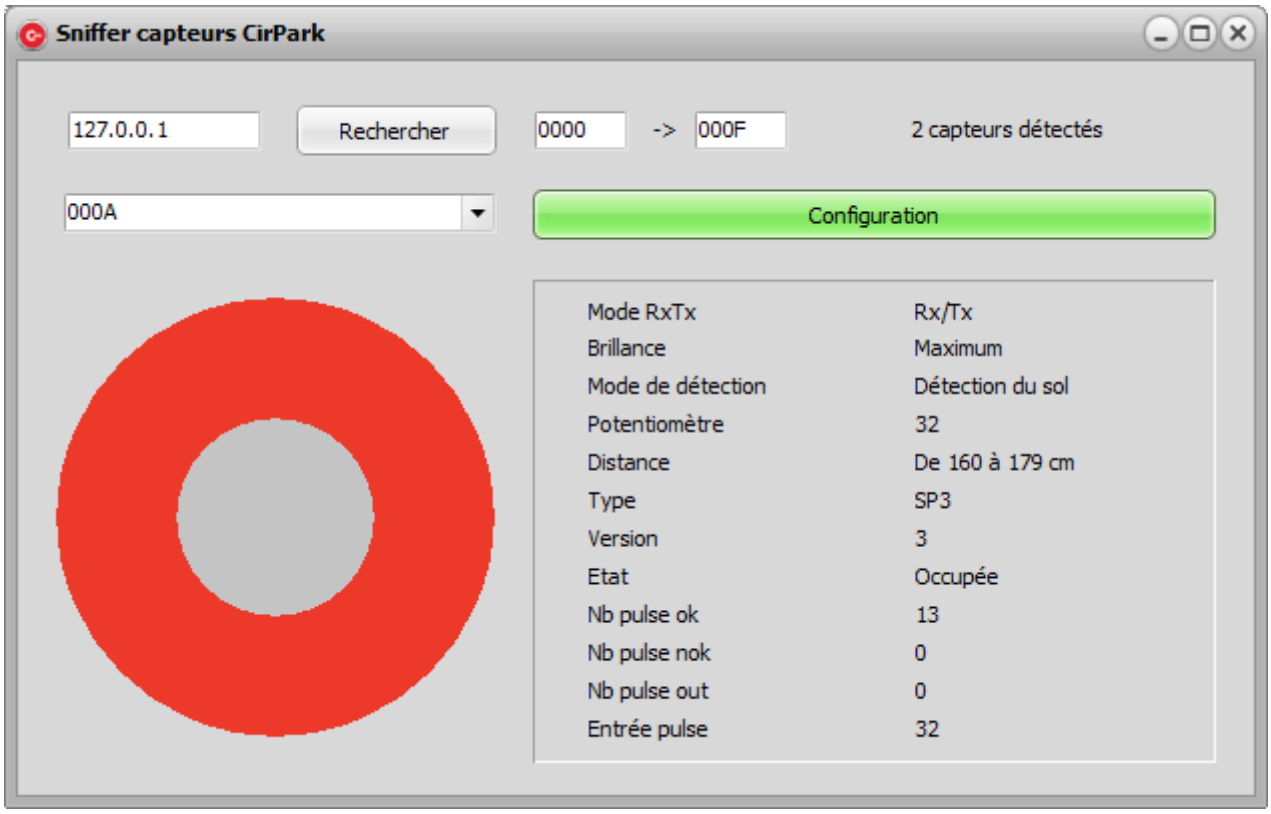


On rappelle la méthode ChaineHexaVersInt() (voir module Exxotest) qu'il est nécessaire d'ajouter à la classe de l'IHM afin de convertir les 4 caractères hexa au format ASCII en nombre :

```
unsigned int TForm1::ChaineHexaVersInt(char *chaineHexa, int nbCar)
{  int nombre=0,j;
   char caractere;
   for(j=0;j<nbCar;j++)
   {   caractere=chaineHexa[j];
         if(caractere>='0' && caractere<='9')
         {  nombre=nombre+((caractere-'0')<<(4*(nbCar-j-1)));
         }
         else nombre=nombre+((caractere-'A'+10)<<(4*(nbCar-j-1)));
   }
   return nombre;
}
```

Lecture et affichage graphique de la configuration d'un capteur

Placer dans la fenêtre de conception les éléments nécessaires à l'affichage graphique des paramètres d'un capteur.



Lors de l'appui sur le bouton Configuration, appeler la méthode LireConfigurationCapteur de l'objet sniffer, en lui passant en paramètre :

```
ChaineHexaVersInt(AnsiString(ComboBox1->Text).c_str(),4)
```

Affecter à chaque Caption des Labels les paramètres retournées par les méthodes de l'objet sniffer :

```
 LabelN->Caption=sniffer.Mode().c_str(); //N étant le numéro du label
...
```

L'anneau rouge est composé de 2 Shapes superposés. Selon l'état du capteur (sniffer.Etat().c_str()), modifier le paramètre Shape1->Brush->Color afin de le faire passer du rouge (occupé) au vert (libre).

TP
Codage d'un thread

Lors de la recherche d'un capteur, l'application ne répond plus au système d'exploitation. Il est nécessaire de placer la code de la recherche (les requêtes UDP dans un thread.

Qu'est ce qu'un thread ?

Créer la classe thread : Fichier->Nouveau->Autre...->Objet thread : classe ThRecherche. Une nouvelle unité (unit2) est créée avec son .h et son .cpp.

Afin que les 2 classes communiquent entre elles, inclure unit2,h dans le fichier unit1,h et unit1,h dans unit2.cpp. Dans unit1.h, ajouter à TForm1 l'attribut ThRecherche *maThread.

Placer dans la méthode Execute() de la classe thread le code du bouton Rechercher de l'interface graphique. Il faudra ajouter devant chaque élément de la classe TForm1 : Form1→. Exemple :

```
Form1->sniffer.ParametrerLeServeurCirpark(AnsiString(Form1->Edit1->Text).c_str(),10001);
```

Tester le fonctionnement : l'application graphique ne se fige plus.

Versionner le code complet.

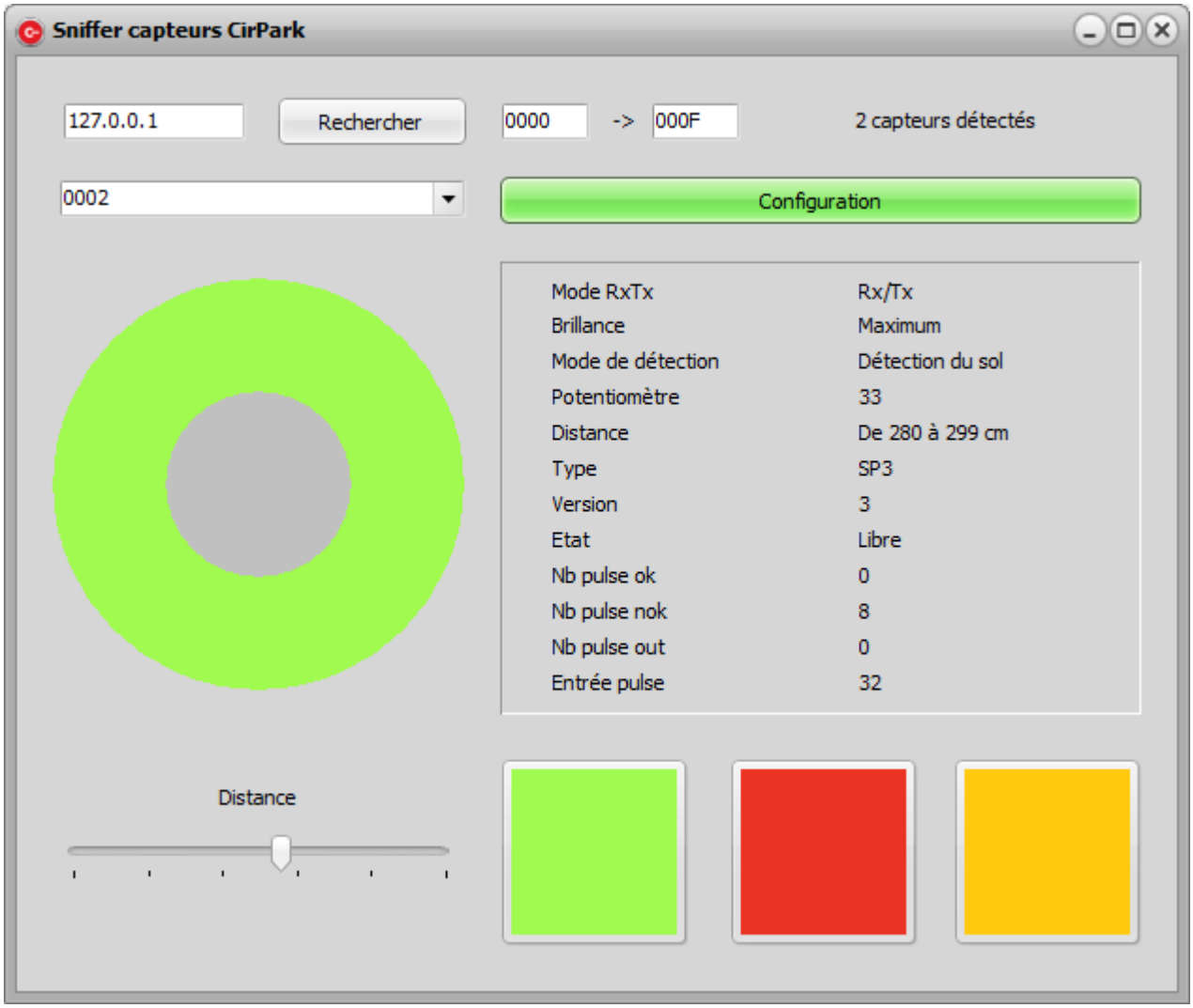
TP
Modification des paramètres des capteurs

La méthode EnvoyerRequeteCapteur de la classe IRSnifferCirpark permet de modifier la configuration d'un capteur, et notamment de changer sa couleur ou bien sa distance de détection.

Quels codes fonction permet de changer la couleur du capteur ?

Quel code fonction permet de modifier la distance de détection ?

Mettre en place l'interface finale : les boutons colorés sont des SpeedButton.



Coder l'envoi des 4 requêtes (vert, rouge, orange, distance).

On donne :

```
if(TrackBar1->Position<160)donnee=0x08;
    else if(TrackBar1->Position<180)donnee=0x09;
    else if(TrackBar1->Position<190)donnee=0x0A;
    else if(TrackBar1->Position<210)donnee=0x0B;
    else if(TrackBar1->Position<230)donnee=0x0C;
    else if(TrackBar1->Position<250)donnee=0x0D;
    else if(TrackBar1->Position<270)donnee=0x0E;
    else if(TrackBar1->Position<280)donnee=0x0F;
    else if(TrackBar1->Position<300)donnee=0x10;
    else if(TrackBar1->Position<320)donnee=0x11;
    else if(TrackBar1->Position<340)donnee=0x12;
    else if(TrackBar1->Position<360)donnee=0x13;
    else if(TrackBar1->Position<380)donnee=0x14;
    else if(TrackBar1->Position<400)donnee=0x15;
    else donnee=0x16;
```

Versionner le code complet.

Diagramme de classe complet

MODULE 06

SÉANCE SYSTÈME 08

TP D'INFORMATIQUE

Durée 2h30

INTERFACE GRAPHIQUE : SUPERVISION DU PARKING EN TEMPS RÉEL

BLOC DE COMPÉTENCES

U6 - VALORISATION DE LA DONNÉE ET CYBERSÉCURITÉ

COMPÉTENCE(S)

C08 - CODER

OBJECTIFS PÉDAGOGIQUES

Codage d'une interface graphique munie de curseurs (scrollbar) permettant de piloter des appareils d'éclairage de scène.

CONNAISSANCES ISSUES DU RÉFÉRENTIEL

- Langages de développement, de description, de création d'API et les IDE associés — Niveau 3
- Programmation multitâche — Niveau 3
- Programmation orientée objet — Niveau 3

CONNAISSANCES OPÉRATIONNALISÉES

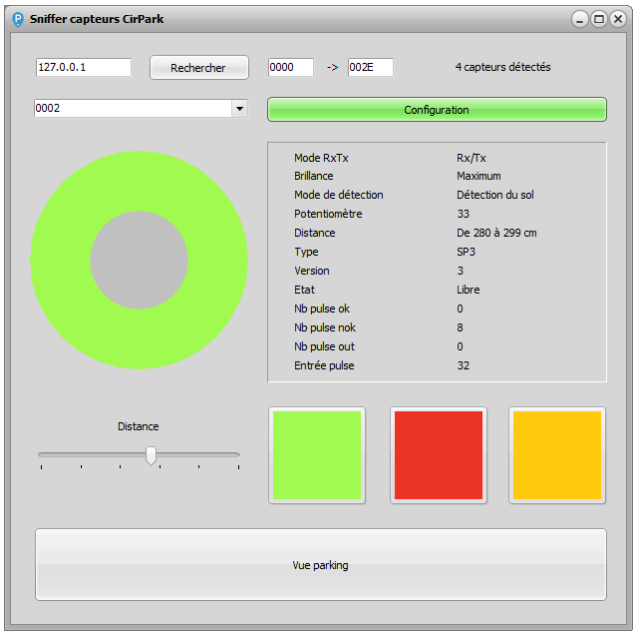
- Concevoir une interface graphique sous Windows — Niveau 2
- Codage d'un thread en C++ — Niveau 2
- Versionner un code — Niveau 2

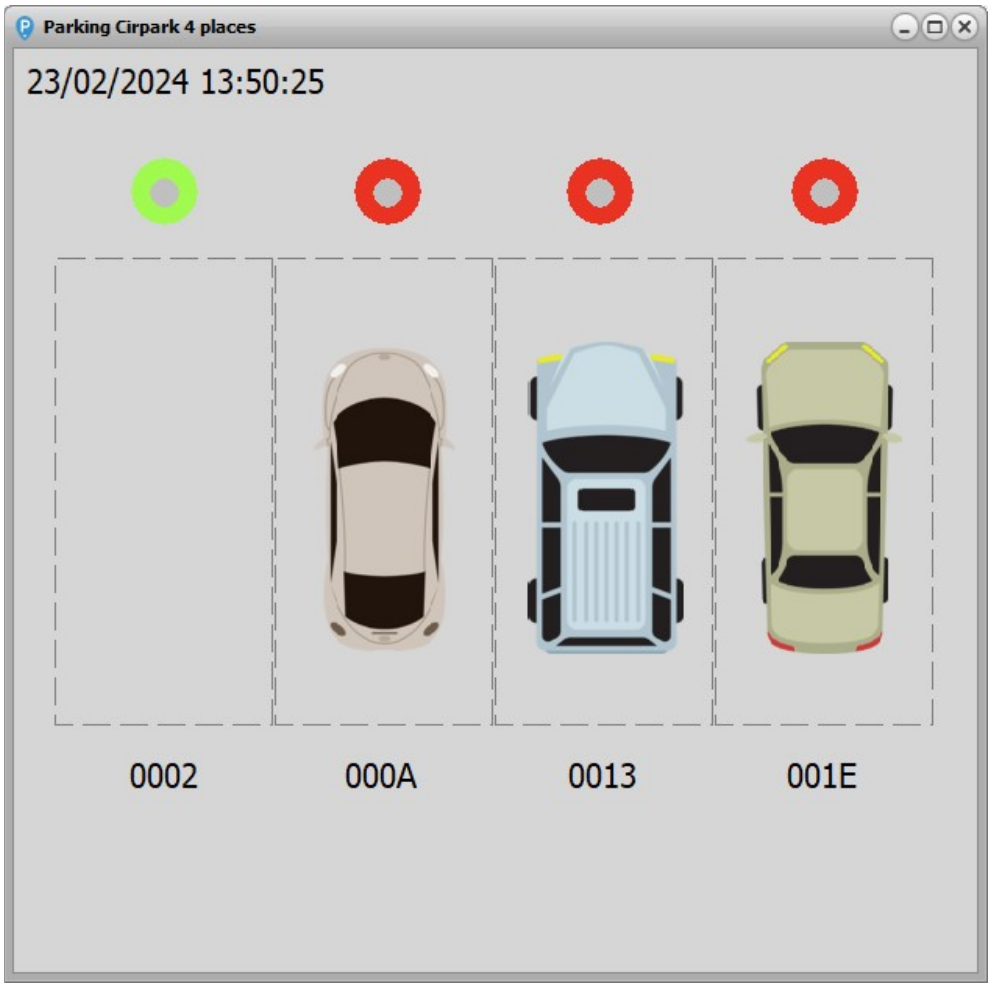
Gaëtan GEORGES
Pascal HAGOT
David IERVESE

Lycée Louis Armand
173 Bd de Strasbourg
Nogent sur Marne

TP

Nouvelle fiche : vue du parking





Conception de la vue du parking

Ajouter une nouvelle fenêtre de conception qui contiendra la vue du parking : Fichier -> nouveau -> Fiche VCL. Une nouvelle classe TForm3 est créée avec son .h (unit3,h) et son .cpp (unit3,cpp). Inclure unit3.h dans unit1.cpp et unit1,h dans unit3.cpp.

Ajouter dans la partie conception de TForm3 les objets graphiques permettant de visualiser la parking de 4 places :

- 2x4 TShapes ronds pour la représentation des capteurs
- 4 TShapes rectangulaires en pointillé pour les emplacements
- 4 TImages pour les véhicules : y charger 4 des images de véhicules fournies dans le répertoire Ressources.
- 4 TLabels pour les numéros des capteurs
- 1 Tlabel pour la date et l'heure

Passage de la fenêtre de recherche des capteurs à la vue du parking

Ajouter le bouton « Vue parking » dans la fenêtre de recherche des capteurs. Dans l'événement OnClick de ce bouton, appeler la méthode Show() de l'objet Form3.

Lors du click sur l'1 des quatre numéros de capteur de la vue parking (unit3,cpp), recopier le Caption du Label dans le Text du Combobox de la fenêtre de recherche. Par exemple :

```
Form1->ComboBox1->Text=Label1->Caption;
```

Appeler alors l'événement OnClick du chargement de la configuration d'un capteur : Par exemple :

```
Form1->Button2Click(this).
```

Appeler enfin la méthode Hide().

L'interface permet maintenant de rechercher les capteurs puis de visualiser le parking. Dans la vue parking, un clic sur le numéro d'un capteur permet de visualiser sa configuration.

TP
Actualisation de la vue parking en « temps réel » : timer de scrutation de l'état des capteurs

Un timer est un thread qui s'exécute selon une cadence prédéfinie.

Affichage de la date et de l'heure

Ajouter un TTimer (désactivé au démarrage) dans la fenêtre de conception de la vue parking. Dans l'événement OnClick du bouton « Configuration » de la fenêtre de recherche des capteurs : activer le timer de la vue parking.

Dans l'événement OnTimer (qu'il faut générer en double-cliquant sur l'objet graphique TTimer), affecter au caption du label date et heure : Date()+Time(). Tester et vérifier la présence et la mise à jour de la date et de l'heure.

Envoi de la requête « état du capteur » : place libre ou bien occupée

Quel code fonction permet de connaître l'état du capteur (libre ou occupée) ?

Quel est l'indice de l'octet de la réponse du capteur qui contient l'état du capteur ?

Quelle méthode de la classe IRSnifferCirpark permet de lire la réponse du capteur ?

Dans l'événement OnTimer, pour chaque label (numéro du capteur), envoyer la requête en donnant le numéro du capteur (utiliser ChaineHexVersInt()). Récupérer la réponse dans un pointeur sur char. Par exemple :

char * reponse=Form1->sniffer.Reponse();

Selon la valeur de l'octet « état » du tableau réponse, rendre l'image du véhicule visible ou non. De la même manière, modifier la couleur du Tschap correspondant au capteur.

Bonus : finalisation de l'application

Ajouter un icône à l'application et modifier l'apparence de l'application : dans les options du projet :

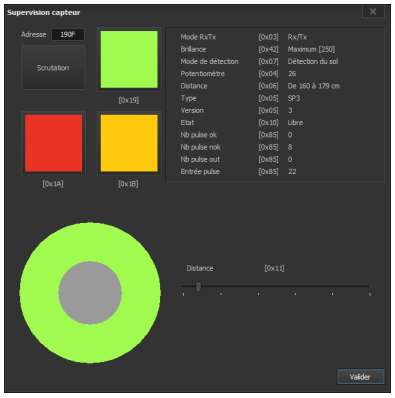
- Application, modifier l'apparence...
- Icone : choisir un icone (flaticon.com), convertir en icône 128x128 (site pgn to ico).

Rendre l'application indépendante de C++ Builder :

- Lieur : décocher Lier avec RTL dynamique,
- Packages d'exécution : décocher,

Versionner le code complet.

TP Bonus
Supervision d'un parking de plusieurs étages
- 1 à 28 places par étage -



MODULE 06
SÉANCE WEB 01
TP D'INFORMATIQUE
Durée 2h30

Installer et exploiter un gestionnaire de place de parking

BLOC DE COMPÉTENCES
U5 – EXPLOITATION ET MAINTENANCE D'UN RÉSEAU INFORMATIQUE
U6 – VALORISATION DE LA DONNÉE ET CYBERSÉCURITÉ

COMPÉTENCE(S)
C09 – INSTALLER UN RÉSEAU INFORMATIQUE
C10 – EXPLOITER UN RÉSEAU INFORMATIQUE
C11 – MAINTENIR UN RÉSEAU INFORMATIQUE

OBJECTIF PÉDAGOGIQUE
Récupérer la configuration d'un système et la modifier en y intégrant un affichage graphique des capteurs et du plan du parking .

CONNAISSANCES ISSUES DU RÉFÉRENTIEL

• Réseaux de terrain	Niveau 4
• Infrastructures matérielles	Niveau 3
• Outillage nécessaire au diagnostic, à la réparation et les équipements de rechange	Niveau 3

CONNAISSANCES OPÉRATIONNALISÉES

• Caractéristiques des capteurs présents	Niveau 3

Gaëtan GEORGES
Pascal HAGOT
David IERVESE

Lycée Louis Armand
173 Bd de Strasbourg
Nogent sur Marne

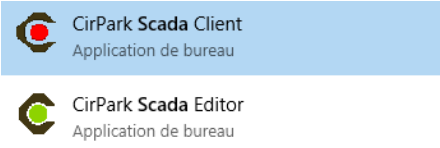
TP

Lors d'une intervention, un technicien doit dans un premier temps récupérer la configuration du système en place avant d'y apporter des modifications. Pour cela vous devez connecter le logiciel scada au poste du prof et importer localement la configuration.

Préparation

Installation

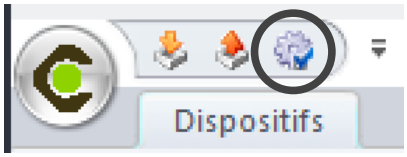
Télécharger puis installer, depuis le disque « L », le logiciel « Cirpark 4.5.3-rc1 » dans le dossier Parking



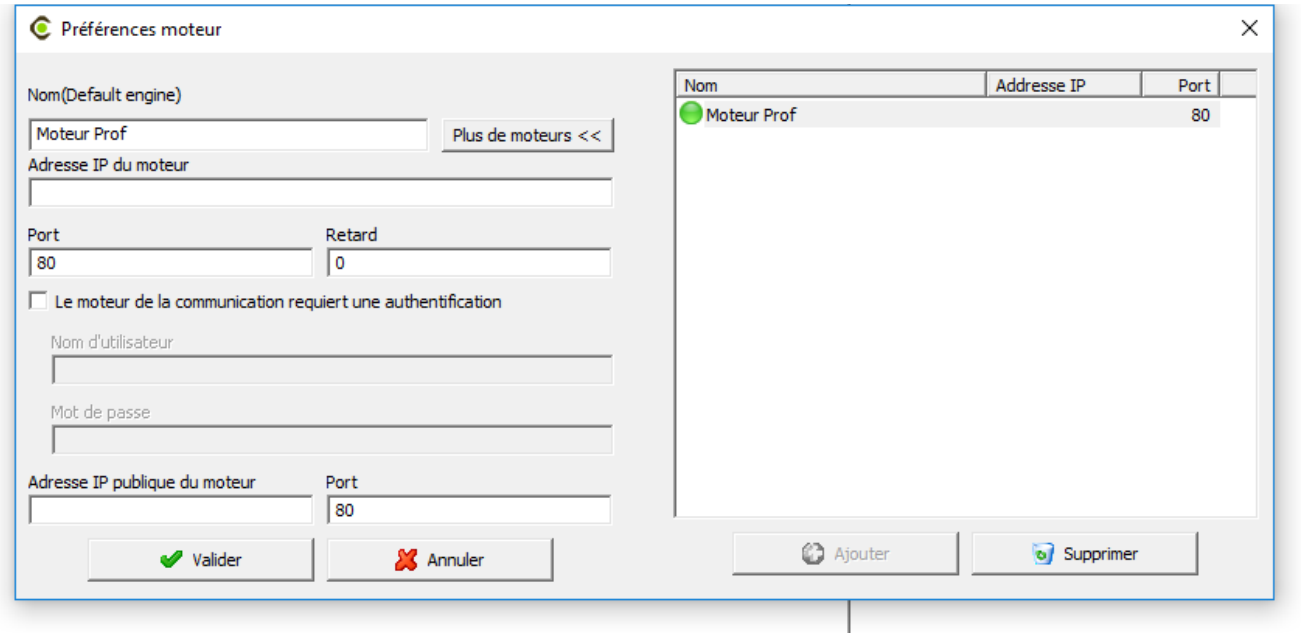
Vérifier l'installation en démarrant l'application « Cirpark Scada Editor »

Configuration

Appuyer sur l'icône de l'engrenage (en haut à gauche)



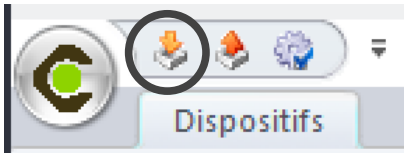
Modifier l'adresse IP pour ajouter le moteur du poste prof



Vérifier la bonne connexion. Une pastille verte doit être présente au niveau du « moteur prof »

Importation

Cliquer sur l'icône d'importation



Importer la configuration.

Configuration locale

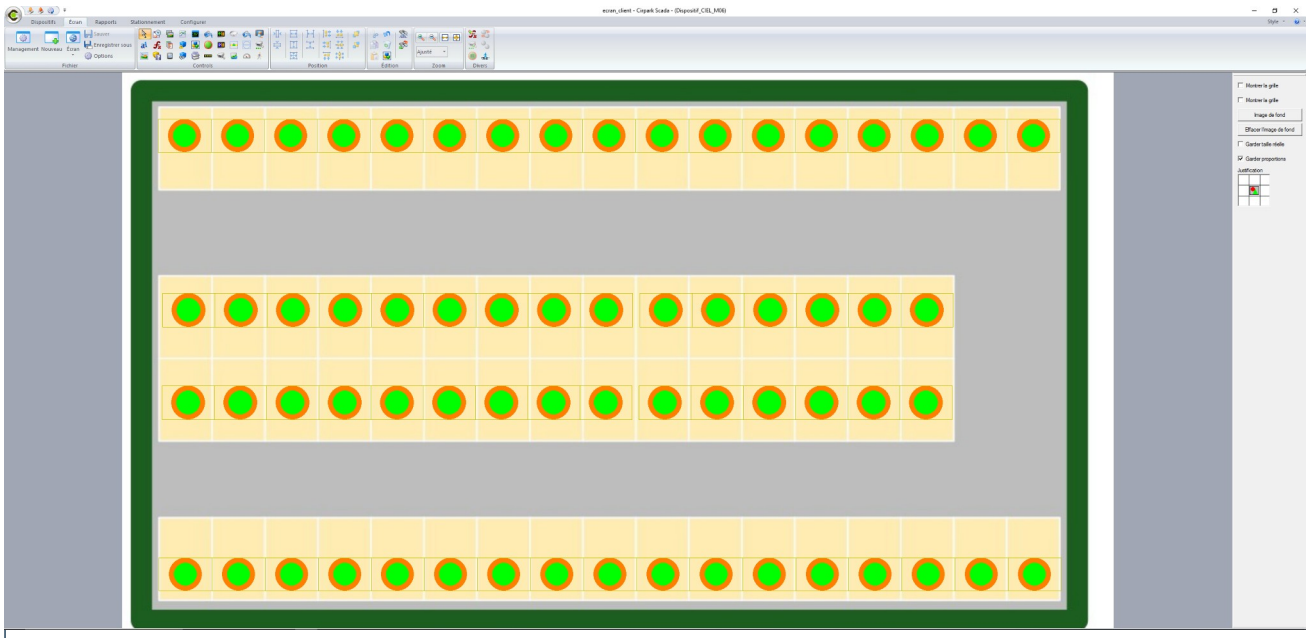
Lancer maintenant l'application « Scada Engine » sur votre machine

Modifier l'engine de « Scada Editor » pour être en local en utilisant l'adresse « 127.0.0.1 »

Créer un écran client

Scada editor

L'objectif de cette partie est de créer un écran client représentant l'image d'un parking avec des places et y intégrer les différents capteurs.



Aller dans le menu « Ecran » de « Cirpark scada editor »

Créer un écran client 1058 x 595 et ajouter une image de fond « parking-CIEL »

Ajouter les capteurs sur chaque places (les doubler si besoin)

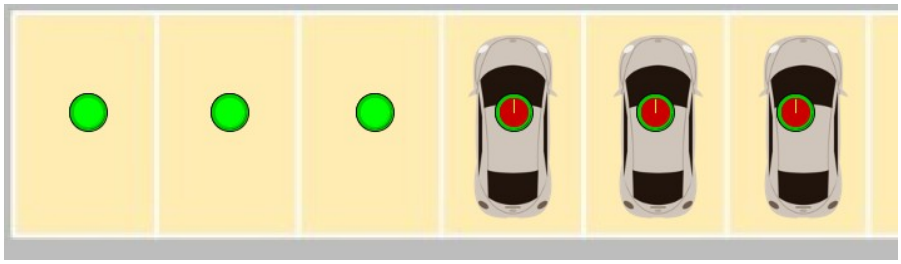
Scada client

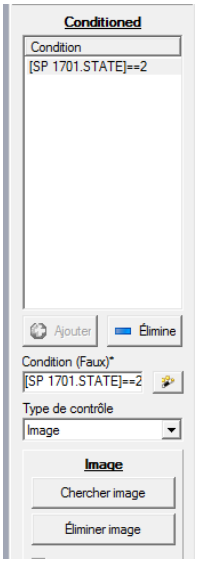
Ouvrir l'application « Scada Client »
Aller dans l'onglet général puis connecter à l'adresse ip de l'engine du prof
Afficher l'écran que vous venez de créer

Bonus :

Vous pouvez ajouter des un objet « conditioned » dans l'écran.

Si le l'état du capteur vaut 2, j'affiche une image de fond avec les voitures





MODULE 06

SÉANCE WEB 02

TP D'INFORMATIQUE

Durée 2h30

Mettre en place la base de données

BLOC DE COMPÉTENCES

U6 – VALORISATION DE LA DONNÉE ET CYBERSÉCURITÉ

COMPÉTENCE(S)

C08 – CODER

OBJECTIF PÉDAGOGIQUE

Mettre en place une base de données gérant les différents capteurs. Importer puis générer des données pour la base de données.

CONNAISSANCES ISSUES DU RÉFÉRENTIEL

• Langages de développement, de description, de création d'API et les IDE associés	Niveau 4
• Chaînes de développements (ordinateur, embarqué, cross compilation)	Niveau 3
• Bases de données et les environnements de traitement de données	Niveau 3

CONNAISSANCES OPÉRATIONNALISÉES

• Installer une base de données	Niveau 3
• Comprendre la structure d'une base de données et les requêtes SQL	Niveau 2

Gaëtan GEORGES
Pascal HAGOT
David IERVESE

Lycée Louis Armand
173 Bd de Strasbourg
Nogent sur Marne

TD
Analyse du besoin

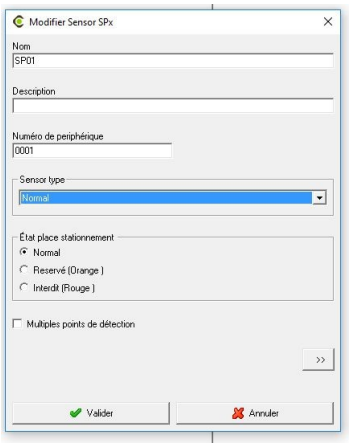
La gestion d'un parking nécessite la mise en place d'un site web qui permettra de visualiser l'état du parking ainsi que l'historique des capteurs.

Pour la suite du projet, nous avons besoin de stocker les informations des capteurs, leurs configurations et leur état dans une base de données.

Structurer la base de données

Donner la liste des tables :

En ce qui concerne les informations des capteurs, le technicien nous transmet l'image suivante à partir de laquelle vous devez constituer les différents champs de votre table



Lister les différents champs possibles

De plus le technicien souhaite avoir plus de données sur la configuration et nous envoie la photo suivante :

Lister les différents champs possibles

Pour avoir un historique du mouvement des places et du parking, vous décidez de stocker l'état de chaque capteur dans une nouvelle table.

Lister les différents champs possibles

TP
Mettre en place la bdd

Modification du service mysql

Connecter vous au serveur et créer une nouvelle base de données (« cirpark-votreNom ») en utilisant la procédure du module précédent

Vérifier que votre utilisateur peut s'y connecter dans phpmyadmin

Table « capteur »

Création de la table capteur

Créer la table « **capteur** » en utilisant la procédure de votre choix (phpmyadmin ou requête sql ou MysqlWorkbench)

Importation des données

Récupérer le fichier capteur.csv depuis le serveur

vérifier l'ordre et le contenu du fichier

Importer le fichier dans la table capteur de votre base de données

Table « configuration »

Mise en place de la table

Modifier votre base de données en y ajoutant la table configuration précédente

La relier à la table capteur

Table « etat »

Mise en place de la table

Modifier votre base de données en y ajoutant la table configuration précédente

La relier à la table capteur

MODULE 06
SÉANCE WEB 03
TP D'INFORMATIQUE
Durée 2h30

CRÉER UN CLIENT UDP EN UTILISANT LE PHP

BLOC DE COMPÉTENCES

U5 – EXPLOITATION ET MAINTENANCE DE RÉSEAU INFORMATIQUE
U6 - VALORISATION DE LA DONNÉE ET CYBERSÉCURITÉ

COMPÉTENCE(S)

C06 - VALIDER UN SYSTÈME INFORMATIQUE
C08 – CODER

OBJECTIF PÉDAGOGIQUE

Créer un client UDP permettant de communiquer avec le système Cirpark

CONNAISSANCES ISSUES DU RÉFÉRENTIEL

- Réseaux informatiques — Niveau 4
- Chaînes de développements (ordinateur, embarqué, cross compilation) — Niveau 3
- Programmation réseau — Niveau 3

CONNAISSANCES OPÉRATIONNALISÉES

- Exploitation du langage PHP — Niveau 2
- Versionner un code — Niveau 2

Gaëtan GEORGES
Pascal HAGOT
David IERVESE

Lycée Louis Armand
173 Bd de Strasbourg
Nogent sur Marne

TD
Analyse du protocole du système

Document technique

Nous utiliserons uniquement des capteurs SP3

> Quel est le bauderate pour un capteur dont la version est 1.4 ?

19200

> Quelle est la configuration du port série ?

8 N 1

> Quel est la structure générale de la trame envoyée par le maître ?

<adr><fnc><bcc>

> Quel est la structure de la trame envoyée pour communiquer avec un capteur SP3 ?

<adrh><adrl><fnc><bcc>

> A quoi sert le bcc ?

Vérifier l'intégrité des données

> Comment le bcc est calculé lors d'une requête ?

<adrh>+<adrl>+<fnc>

> Donner le calcule du bcc pour la réponse :

~(<adrh>+<adrl>+<fnc>)

> A quoi sert la partie <fnc> ?

Cela correspond aux codes fonctions de la demande

Analyse des codes fonctions

Les codes fonctions permettent de définir l'objet de la demande

> Quel code fonction permet de connaître l'état « libre ou occupé » du capteur ?

0x10

> Quel code fonction permet de connaître la distance de détection du capteur ?

0x06

> Quelles sont les valeurs possibles du code fonction précédent

> A quoi sert le code fonction 0x42

Demande sur la valeur de la luminosité

> Quel code fonction permet de modifier la luminosité

0x41

> Quelle valeur faut il envoyer pour fixer la luminosité à 50 %

TP
Communication UDP en PHP

La classe « socket » en PHP

La mise en place d'une communication UDP nécessite l'utilisation de socket. En PHP, il existe également la classe socket.

Voici le code PHP que nous utiliserons pour communiquer en UDP avec le système Circontrol

```
$capteur=str_split("1700",2);
$adrh=$capteur[0];
$adrl=$capteur[1];
$codeFonction="10";
$bcc=dechex(hexdec($adrh)+hexdec($adrl)+hexdec($codeFonction));  // Addition decimale puis conversion hexa

$ipServeur = "192.168.1.58";
$port = "10001";

$message=hex2bin($adrh.$adrl.$codeFonction.$bcc);
echo "message envoyé : ".$adrh.$adrl.$codeFonction.$bcc;

$sock = socket_create(AF_INET, SOCK_DGRAM, 0);

//Send the message to the server
socket_sendto($sock, $message , strlen($message) , 0 , $ipServeur , $port);

//Now receive reply from server and print it
socket_recvfrom ( $sock , $reponse , 2045 , MSG_WAITALL, $ipServeur , $port );
```

En recherchant en ligne, répondez aux questions suivantes :

A quoi sert la fonction socket_creat() ?

Donner la signification des des paramètres « AF_INET,SOCKE_DGRAM »

Lister les différentes méthodes permettant d'envoyer un message avec la classe socket de PHP :

socket_send,socket_sendmsg, socket_sendto

Quelle est la différence entre « socket_sendto et socket_send » ?

Lister les différentes méthodes permettant de recevoir un message avec la classe socket de PHP :

socket_recv,socket_

Quelle est la différence entre « socket_recv et socket_recvfrom » ?

Mise en place du code

Créer un fichier cirpark_client_udp.php

Recopier le code PHP permettant d'envoyer une requête UDP

Modifier le code avec les variables

```
$ipServeur=ip_du_poste_prof;
$port=numero_port_serveur_prof;
```

Tester le code

Pour tester le code nous allons nous connecter au serveur en ssh et exécuter le fichier php en local avec la commande suivante :

```
php cirpark_client_udp.php
```

TP
Traitement de la réponse

Afin d'exploiter la réponse et de l'enregistrer dans la base de données, nous allons extraire les données depuis la chaîne de caractères « $reponse » et de les insérer dans un tableau « $tableauData ».

Pour cela nous allons utiliser le code suivant :

```
$tableauData=str_split(bin2hex($reponse),2)
```

A quoi sert la fonction str_split ?

Expliquer les deux paramètres utilisés dans le code précédent.

Compléter le fichier cirpark_client_udp.php avec la variable « $tableauData »

Compléter le code avec la possibilité d'afficher « Libre » ou « Occupée » en fonction de la réponse du capteur

MODULE 06
SÉANCE WEB 04
TP D'INFORMATIQUE
Durée 2h30

MISE A JOUR DE LA BASE DE DONNÉES DEPUIS LE CLIENT UDP

BLOC DE COMPÉTENCES
U6 – VALORISATION DE LA DONNÉE ET CYBERSÉCURITÉ

COMPÉTENCE(S)
C08 – CODER

OBJECTIF PÉDAGOGIQUE
Créer une boucle permettant d'interroger un serveur puis mettre à jour la base de données en utilisant le langage PHP.

CONNAISSANCES ISSUES DU RÉFÉRENTIEL

• Langages de développement, de description, de création d'API et les IDE associés	Niveau 4
• Chaînes de développements (ordinateur, embarqué, cross compilation)	Niveau 3
• Bases de données et les environnements de traitement de données	Niveau 3
• Outils logiciels d'évaluation, de traçabilité de l'information	Niveau 3

CONNAISSANCES OPÉRATIONNALISÉES

• Mise en place de requêtes SQL	Niveau 3
• Analyser une communication réseau avec Wireshark	Niveau 3
• Versionner un code	Niveau 3

Gaëtan GEORGES
Pascal HAGOT
David IERVESE

Lycée Louis Armand
173 Bd de Strasbourg
Nogent sur Marne

TP

Liste des capteurs

L'objectif est de récupérer la liste des capteurs présents dans la base de données

> Donner la requête SQL permettant de récupérer la liste des idcapteurs ainsi que la le numéro de la table capteur

```
SELECT idcapteur,numero FROM capteur
```

> Compléter le fichier « cirpark_client_UDP.php » avec le code nécessaire pour récupérer la liste des numéros des capteurs en utilisant les requêtes préparées (cf. Module M05).

Interroger les capteurs

> Faire une boucle et pour chaque capteur, envoyer un message UDP au serveur afin de connaître l'état du capteur

Insérer dans la base de données

> Donner la requête permettant d'insérer l'état, la date et l'idcapteur dans la table etat

```
INSERT INTO etat(etat,date,idcapteur) values(?,?,?)
```

> Compléter le fichier « cirpark_client_UDP.php » avec les conditions suivantes

```
Si le numéro de capteur est présent et que sa valeur a changée alors on insère une nouvelle valeur
dans la table etat
```

> Exécuter le fichier php depuis le serveur
> Vérifier dans PHPmyadmin le bon fonctionnement
> Faire la même chose pour la configuration des capteurs

Bonus : Mise à jour automatique

> Ajouter une boucle while(1) et un sleep d'une minutes pour interroger le serveur toutes les minutes
> Lancer le fichier « cirpark_client_UDP.php » directement sur le serveur et depuis votre dossier M06 avec la commande suivante :

```
php cirpark_client_UDP.php
```

> Vérifier le bon fonctionnement avec wireshark

MODULE 06
SÉANCE WEB 05
TP D'INFORMATIQUE
Durée 2h30

MISE EN PLACE D'UNE API REST EN PHP

BLOC DE COMPÉTENCES
U6 – VALORISATION DE LA DONNÉE ET CYBERSÉCURITÉ

COMPÉTENCE(S)
C08 – CODER

OBJECTIF PÉDAGOGIQUE
Mettre en place une API REST en PHP, permettant de récupérer les valeurs de vitesse et de régime stockées dans la base de données. Utilisation d'un client REST pour tester l'API. Vérifier les requêtes HTTP en utilisant Wireshark.

CONNAISSANCES ISSUES DU RÉFÉRENTIEL

• Langages de développement, de description, de création d'API et les IDE associés	Niveau 4
• Chaînes de développements (ordinateur, embarqué, cross compilation)	Niveau 3
• Bases de données et les environnements de traitement de données	Niveau 3
• Outils logiciels d'évaluation, de traçabilité de l'information	Niveau 3

CONNAISSANCES OPÉRATIONNALISÉES

• Mise en place d'une API REST en PHP	Niveau 2
• Analyser une communication réseau avec Wireshark	Niveau 3
• Versionner un code	Niveau 3

Gaëtan GEORGES
Pascal HAGOT
David IERVESE

Lycée Louis Armand
173 Bd de Strasbourg
Nogent sur Marne

TD
Préparation de l'API REST

Objectif

Mettre en place la liste des différentes requêtes HTTP de l'API REST

API

Notre API devra fournir les éléments suivants :

- Récupérer le nom, le type,le numero,l'etat, la hauteur, l'éclairage de tous les capteurs
- Récupérer l'état ou la configuration de tous les capteurs
- Récupérer l'état ou la configuration d'un capteur connaissant son numéro

> Lister les 5 différentes requêtes HTTP qui seront présentes dans notre API

http://...../rest.php/capteur Récupérer la liste de tous les capteurs
http://...../rest.php/capteur/etat Les états de tous les capteurs
http://...../rest.php/capteur/configuration La configuration de tous les capteurs
http://...../rest.php/capteur/etat/{numero} L'etat d'un capteur dont le numero est fourni
http://...../rest.php/capteur/configuration/{numero} La configuration d'un capteur

Documentation

Une documentation API permet à l'utilisateur de connaître les différentes requêtes possibles ainsi que les réponses associées. Vous allez créer un document en vous basant de l'exemple suivant :

Requête 1 :

Requête :
GET rest.php/capteur
Réponse :

```
{"status":"success",
 "valeur": [{"nom":"SP1601",
            "type":"Normale",
            "numero":"1601",
            "etat":"Occupée",
            "hauteur":"280",
            "eclairage":"250"
           },....
          ]
}
```

SQL : Select

SELECT capteur.nom, capteur.type, capteur.numero, configuration.hauteur, configuration.eclairage FROM capteur INNER JOIN configuration ON capteur.id=configuration.id_capteur

Requête 2 :

Requête:
GET rest.php/capteur/etat
SQL : SELECT numero,etat FROM etat,capteur where capteur.idcapteur=etat.idcapteur

Requête 3 :

Requête:
GET rest.php/capteur/configuration
SQL : SELECT * FROM configuration,capteur where capteur.idcapteur=configuration.idcapteur

Requête 4 :

Requête :
GET rest.php/capteur/etat/{numero}
SQL : SELECT etat FROM etat,capteur where capteur.idcapteur=etat.idcapteur and capteur.numero=?

Requête 5 :

Requête:
GET rest.php/capteur/configuration/{numero}
SQL : SELECT * FROM configuration,capteur where capteur.idcapteur=configuration.idcapteur and capteur.numero=?

TP
Mettre en place l’API REST

Créer le fichier rest.php

En vous aidant du module M05, compléter le fichier rest.php avec les requêtes précédentes

Vérifier le bon fonctionnement en utilisant l’extension REST d’un navigateur

MODULE 06

SÉANCE WEB 06

TP D'INFORMATIQUE

Durée 2h30

AFFICHAGE DE L'ÉTAT DES CAPTEURS DANS UNE PAGE WEB : PLAN DE PARKING

BLOC DE COMPÉTENCES

U6 – VALORISATION DE LA DONNÉE ET CYBERSÉCURITÉ

COMPÉTENCE(S)

C08 – CODER

OBJECTIF PÉDAGOGIQUE

Utilisation de l'API REST EN JS : envoi d'une requête HTTP GET. Affichage des données sous forme de tableau dans une page Web

CONNAISSANCES ISSUES DU RÉFÉRENTIEL

• Langages de développement, de description, de création d'API et les IDE associés	Niveau 4
• Bases de données et les environnements de traitement de données	Niveau 3
• Chaînes de développements (ordinateur, embarqué, cross compilation)	Niveau 3

CONNAISSANCES OPÉRATIONNALISÉES

• Envoi de requête HTTP en Javascript	Niveau 3
• Versionner un code	Niveau 3

Gaëtan GEORGES
Pascal HAGOT
David IERVESE

Lycée Louis Armand
173 Bd de Strasbourg
Nogent sur Marne

TP
Afficher la valeur des capteurs

Accueil

Créer un fichier index.html

Ajouter une barre de navigation permettant soit d'afficher la liste des capteurs soit d'afficher le plan du parking.

Ajouter les id « planNAV et listeNAV » dans la barre de navigation

Créer le fichier ajax.js

Rendre les éléments de la barre de navigations cliquable avec un addEventListener

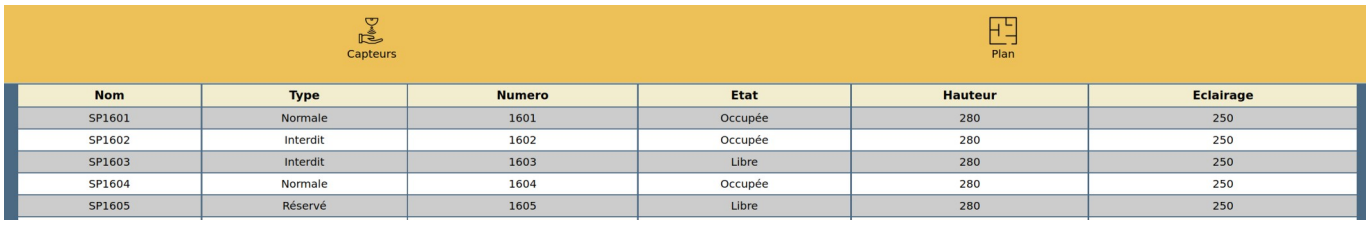
```
document.getElementById('planNAV').addEventListener('click',AfficherPlanHTML);
document.getElementById('capteurNAV').addEventListener('click',AfficherListeHTML);
```

Javascript : ajax.js

Liste des capteurs

Créer la fonction « AfficherListeHTML() »

Cette fonction doit faire appel à la requête rest.php/capteur puis afficher le résultat sous la forme suivante



Nom	Type	Numero	Etat	Hauteur	Eclairage
SP1601	Normale	1601	Occupée	280	250
SP1602	Interdit	1602	Occupée	280	250
SP1603	Interdit	1603	Libre	280	250
SP1604	Normale	1604	Occupée	280	250
SP1605	Réservé	1605	Libre	280	250

Compléter la fonction afin d'avoir le résultat précédent

Créer un page html avec la liste des capteurs, leurs états et leurs luminosités et leurs hauteurs

MODULE 06
SÉANCE WEB 07
TP D'INFORMATIQUE
Durée 2h30

Plan de parking

BLOC DE COMPÉTENCES
U6 – VALORISATION DE LA DONNÉE ET CYBERSÉCURITÉ

COMPÉTENCE(S)
C08 – CODER

OBJECTIF PÉDAGOGIQUE
Utilisation de l'API REST EN JS : envoi d'une requête HTTP GET. Affichage des données sous forme de plan dans une page Web

CONNAISSANCES ISSUES DU RÉFÉRENTIEL

• Langages de développement, de description, de création d'API et les IDE associés	Niveau 4
• Bases de données et les environnements de traitement de données	Niveau 3
• Chaînes de développements (ordinateur, embarqué, cross compilation)	Niveau 3

CONNAISSANCES OPÉRATIONNALISÉES

• Utilisation d'une bibliothèque graphique en JS	Niveau 2
• Versionner un code	Niveau 3

Gaëtan GEORGES
Pascal HAGOT
David IERVESE

Lycée Louis Armand
173 Bd de Strasbourg
Nogent sur Marne

TP
Afficher le plan du parking

Objectif

Créer une plan de parking et y afficher le numéro du capteur ainsi que son état

Plan.html

> Récupérer le fichier plan.html ainsi que le fichier style.css

Javascript

> Créer la fonction « AfficherPlanHTML() » permettant de charger le fichier plan.html en utilisant l'ajax puis d'appeler la fonction « AfficherCapteurListe() »
>
> Créer la fonction AfficherCapteurListe()
>
> Compléter la fonction précédente avec le code permettant de récupérer le numéro des capteurs, leurs états et de les afficher dans les places de parking.

MODULE 06
SÉANCE WEB 08
TP D'INFORMATIQUE
Durée 2h30

SÉCURITÉ

BLOC DE COMPÉTENCES
U5 – EXPLOITATION ET MAINTENANCE DE RÉSEAUX INFORMATIQUES

COMPÉTENCE(S)
C06 – VALIDER UN SYSTÈME INFORMATIQUE

OBJECTIF PÉDAGOGIQUE
Sécurité : Cryptage AES des données provenant de la base de données (CryptoJS.AES.encrypt)

CONNAISSANCES ISSUES DU RÉFÉRENTIEL

• Protocoles usuels IPv4, HTTP, HTTPS,TCP/IP, Ethernet, ipV6, DNS, DHCP, SSH	Niveau 4
• Sécurisation des réseaux (ACL, mots de passe, pare-feu)	Niveau 3
• Outils logiciels d'évaluation, de traçabilité de l'information, de tests, d'analyse de traitement et de rapport de l'incident (C11)	Niveau 3
• Maîtrise des environnements de développement, d'intégration, de déploiement logiciel et des versions logicielles associées	Niveau 3

CONNAISSANCES OPÉRATIONNALISÉES

• Analyse Wireshark de la communication HTTP	Niveau 3
•	
• Versionner un code	Niveau 3

Gaëtan GEORGES
Pascal HAGOT
David IERVESE

Lycée Louis Armand
173 Bd de Strasbourg
Nogent sur Marne

TP
Formulaire de connexion

Objectif :

Limiter l’accès et l’utilisation du site web par un formulaire de connexion

Mettre en place un formulaire de connexion avec la BDD, l’API rest et l’ajax